4

Knowledge Representation

4.1 Introduction

Artificial intelligence as technology has always fascinated human beings. Multiple science fiction novels and movies have demonstrated the use of AI-powered systems (like robots) that can think, act, understand complex information and make smart decisions based on it.

Although this all looks like a fantastic development, we agree that it is not easy to make machines that behave exactly like humans. This is because humans have conscience and this conscience develops gradually in us with our knowledge, experiences and memories.

To build AI systems that have conscience, we need to inculcate knowledge in them. This chapter is, therefore, dedicated towards representing knowledge in machines.

Knowledge and Intelligence

Knowledge of real world plays a crucial role in intelligence and creating AI agents that demonstrate intelligent behaviour. Such an agent acts by sensing its environment and using knowledge to act intelligently. However, for this, the knowledge or experience about the input is mandatory. The relationship between knowledge and intelligence can be clearly understood from **Fig. 4.1**. You can see that if we remove the knowledge component, then the decision maker will not be able to sense the environment accurately and take appropriate decisions.

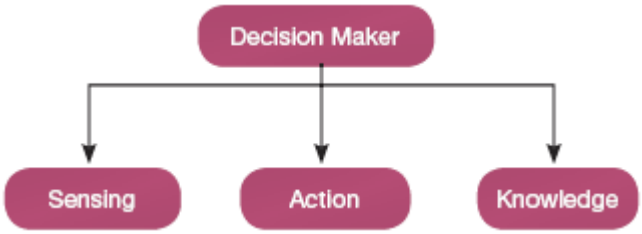


FIGURE 4.1 Relationship between Knowledge and Intelligence

4.2 Knowledge Representation

Humans are best at understanding, reasoning and interpreting things they know (knowledge). Humans use their knowledge to perform various actions in the real world. ***But how machines perform actions, comes under the domain of knowledge representation and reasoning.***

Knowledge representation and reasoning (KR, KRR) is a part of artificial intelligence which focuses on how AI agents think and how their thinking contributes to intelligent behaviour. A knowledge-based system represents information about the real world in such a way that a computer can easily understand and utilize this knowledge to solve complex real-world problems like diagnosing a medical condition or communicating with humans in natural language.

Knowledge representation represents knowledge available in the form of beliefs, intentions and judgments that an intelligent agent must possess for automated reasoning. One of the primary purposes of knowledge representation includes modelling intelligent behaviour for an agent.

Knowledge representation is not just storing data in a database. It goes beyond this aspect to facilitate an intelligent machine to learn from that knowledge and experiences so that it can behave intelligently like a human.

4.2.1 What Knowledge Needs to be Represented?

Humans have intuition, intentions, prejudices, beliefs, judgments, common sense, etc. apart from knowledge about certain facts. We need to incorporate all this information in a machine-understandable format and

make the AI system truly intelligent. For this, we need to represent the following knowledge in AI systems:

1. **Object:** Information and facts about all objects (relevant in context). For example, in a self-driving car, vehicles and roads are objects.
2. **Events:** Information and facts about actions which occur in the real world. For example, in a self-driving car, an event can be applying breaks when an object comes in front of it.
3. **Performance:** The manner in which actions are performed. It describes the behaviour related to how to do things.
4. **Meta-knowledge:** It is the knowledge about knowledge (what we know).
5. **Facts:** Facts are the truths about the real world that needs to be represented for an intelligent agent.
6. **Knowledge base:** It is the most important component of a knowledge-based agent that stores a group of sentences (technical sentences, not simple English language ones).

4.2.2 What is Knowledge?

We know that knowledge is the basic element of our brain as it helps us to know and understand the things logically and is gained by experiences of facts, data and situations. A knowledgeable person performs all the actions in a better way. Everyone has five types of knowledge illustrated in **Fig. 4.2** and enumerated as follows.

1. **Meta knowledge** is the knowledge about knowledge.
2. **Heuristic knowledge** is the knowledge about a specific topic. For example, it can be knowledge of some experts in a filed or subject. Heuristic knowledge is treated as the rule of thumb as it is based on previous experiences and awareness of approaches, which are good to

work but not guaranteed.

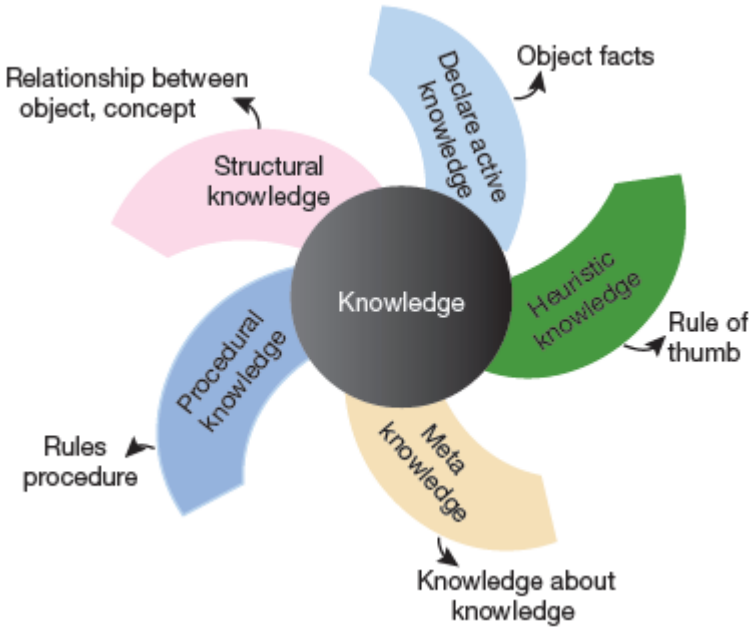


FIGURE 4.2 Types of knowledge

3. **Procedural knowledge,** also known as imperative knowledge, gives information about how to achieve or do something. This knowledge includes rules, strategies, procedures, agendas, etc. that can be directly applied to perform any task.
4. **Declarative knowledge** is the information that we have about an object. This knowledge helps us to describe a particular concept, fact, object and its attributes. Declarative knowledge is simpler than procedural language and is also called descriptive knowledge as it is usually represented using declarative sentences.
5. **Structural knowledge** is the basic knowledge to solve complex problems. It describes relationships between various concepts or objects such as kind of, part of, and grouping of something.

4.2.3 What is Logic?

Logic is the main component of any knowledge as it facilitates a person to draw conclusion by filtering required information from large volumes of sentences.

In AI, knowledge is represented using logic. Every logic has three main elements.

Syntax

Every language has its own syntax to specify the sequence of constructs that makes a complete sentence in that language. It is therefore not wrong to say that syntax is the representation of a language.

Semantics

Semantics is used to check if a syntactically correct sentence is logically correct and meaningful or not. Therefore, semantics defines the sense of the sentence which relates to the real world. For example, consider the statement:

Ram is riding a bike.

This sentence is syntactically as well as semantically correct. However, the sentence,

Bike is riding Ram.

is syntactically correct but semantically incorrect.

Logical Inference

Inference means to deduce conclusions in the context of some fact or problem. Correspondingly, logical inference uses inference algorithms to think all the possible reasons that can give a proper result.

4.2.4 Cycle of Knowledge Representation in AI

An AI system has several components (**Fig. 4.3**) that helps it to exhibit an intelligent behaviour. These components include the following:

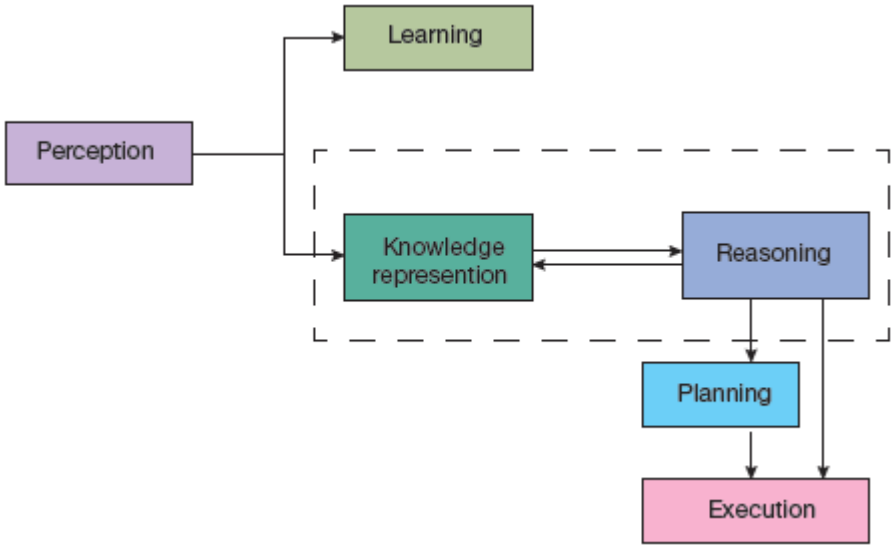


FIGURE 4.3 Components of an AI system

Perception

The perception component retrieves data from the environment, discovers the source(s) of noise, checks if the AI was damaged by anything and defines response to be given when any sense has been detected. Data from the environment is gathered using different types of sensors in the form of video, audio, text, time, temperature or any other sensor-based input.

Learning

The learning component learns from the data that is captured by the perception component. The aim here is to develop a system that can be taught instead of being programmed. With learning, the system focuses on self-improvement through knowledge acquisition, inference, acquisition of heuristics, faster searches, etc.

Knowledge Representation and Reasoning

This is the main component which shows human-like intelligence in the machines. Knowledge representation techniques (refer **Fig. 4.4**) are designed to understand using a top-down approach focussing in detail, what an agent needs to know to behave intelligently. The KRR component also defines how automated reasoning procedures can make this knowledge available as needed.



FIGURE 4.4 Forms of Knowledge Representation

Planning and Execution

This component analyses knowledge representation and reasoning. The planning component selects an initial state, enumerates preconditions and effects, and a sequence of actions to achieve the goal state. Once planning is done, actions are executed to get the desired results.

4.2.5 Knowledge Representation Requirements

A good knowledge representation system must have the following properties.

1. Representational accuracy, to ensure that the system represents all kinds of required knowledge.
2. Inferential adequacy to manipulate the representational structures to produce new knowledge corresponding to the existing structure.
3. Inferential efficiency to direct the inferential knowledge mechanism to generate appropriate results.
4. Acquisitional efficiency that helps the knowledge representation system to easily acquire new knowledge using automatic methods.

4.3 Knowledge-Based Agent

In AI, the agents that mimic a human being's knowledge are known as knowledge-based intelligent agents. Such an agent uses its knowledge and reasoning to act efficiently by making appropriate decisions. A knowledge-based agent, therefore, does the following:

1. It maintains an internal state of knowledge to represent states, actions, etc.
2. It deduces reasoning with that knowledge.
3. It updates their knowledge after observations by incorporating new precepts.
4. It takes actions accordingly.

Knowledge-based agents represent knowledge that can be reasoned to act intelligently according to the requirements. Usually, these agents store knowledge about its surroundings in the form of sentences.

4.3.1 The Architecture of Knowledge-Based Agent

Knowledge-based agents represent the world with some formal representation to act intelligently. For this, they contain a knowledge-base and an inference system.

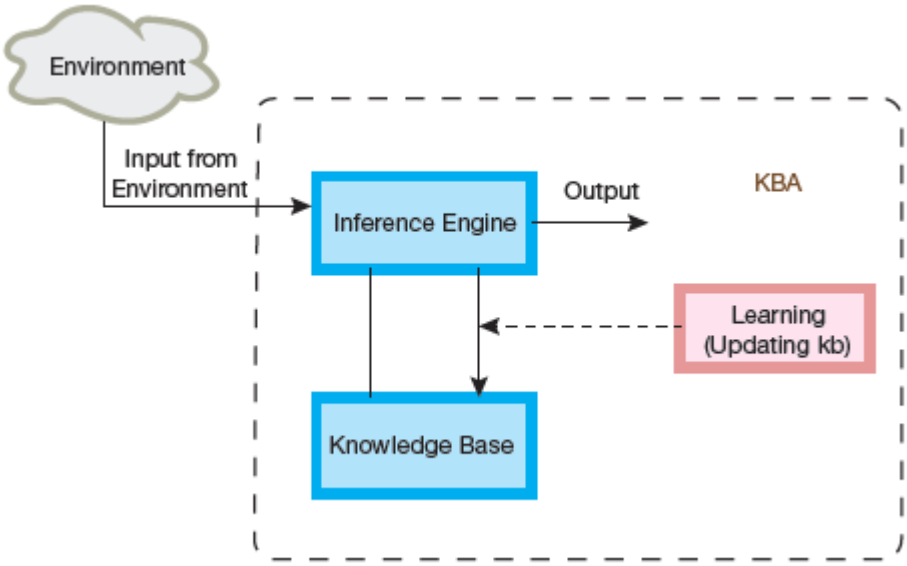


FIGURE 4.5 Architecture of Knowledge-Based Agent

Knowledge base (KB) is the most important component of a knowledge-based agent. It stores real facts of world using sentences which are explained in knowledge representation language.

Inference engine (IE) is the knowledge-based system engine that infers new knowledge in the system.

Figure 4.5 shows a generalized architecture for a knowledge-based agent (KBA). According to **Fig. 4.5**,

Step 1: The KBA perceives the environment to take input from it.

Step 2: This input is taken by the inference engine of KBA.

Step 3: The inference engine interacts with the knowledge base to make decisions based on the knowledge stored in it.

Note that the learning element of the KBA regularly updates the knowledge base by learning new knowledge. This knowledge base is nothing but a collection of sentences (technical information not same as a sentence in English) that are expressed in a language known as knowledge representation language and stores fact about the world.

In AI, a sentence is not an English language sentence, but a fact represented in a language known as knowledge.

The inference system derives new sentences from old ones and also facilitates addition of new sentence(s) to the knowledge base. As stated earlier, a sentence is a proposition about the world. New information is deduced from the knowledge base by applying logical rules (either forward chaining or backward chaining).

4.3.2 Operations Performed By KBA

Whenever the KBA is called to add/update knowledge in the KB or to answer queries made to the KB, it acts intelligently by performing a set of important functions that are given below.

TELL operation tells the knowledge base what knowledge it already has about its environment and what additional knowledge it needs to know. TELL is also used to inform the knowledge base which action was selected and performed by the agent.

ASK operation asks the knowledge base what action it should perform. The agent gets its answer from the knowledge base.

PERFORM operation performs the selected action.

4.3.3 A Generic Knowledge-Based Agent

Given below is the structure outline of a generic knowledge-based agent program (**Fig. 4.6**).

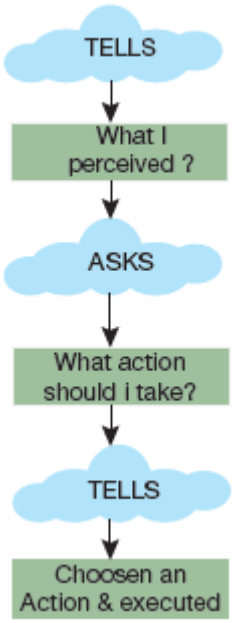


FIGURE 4.6 Mechanism of an agent program

```
function KB-AGENT(percept):
persistent: KB, a knowledge base
            t : time ( counter initially 0)
TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
Action = ASK(KB, MAKE-ACTION-QUERY(t))
TELL(KB, MAKE-ACTION-SENTENCE(action, t))
t = t + 1
return action
```

The knowledge-based agent accepts the information it perceives from the environment as input and returns an action as the output. The agent maintains the knowledge base (KB). It has some background knowledge of the real world and maintains a counter, t with initial value zero to indicate the time for the whole process.

Each time the function is called, the KBA performs operations as given below.

TELLS the KB what it perceives. For this, the MAKE-PERCEPT-SENTENCE function is used to generate a sentence that informs about what information the agent had perceived at the given time.

ASKS KB what action it should take. The MAKE-ACTION-QUERY function is executed to generate a sentence asking which action should be performed at the current time.

TELLS the KB about the action that was chosen. The MAKE-ACTION-SENTENCE function generates a sentence informing about the chosen action that was executed.

4.3.4 Various Levels of Knowledge-Based Agent

A knowledge-based agent can be viewed at different levels as follows.

Knowledge Level

This is the first level of a KBA. In this level, we need to specify what the agent knows, and what its goals would be. This information is then used to fix the behaviour of the agent. For example, if an agent has to go from point A to point B and it knows the optimum way to reach the destination, then this information is added at the knowledge level (using sentences).

Logical Level

At this level, the knowledge representation of the knowledge stored in the knowledge base is understood. For this, sentences are encoded into different logics. Therefore, knowledge is encoded into logical sentences. Continuing with the above example, agent deduces (understands) logic to reach destination, that is, point B.

Implementation Level

This is the physical representation of logic and knowledge. At this level, the agent performs actions according to the knowledge obtained from the

logical and knowledge level. For example, the agent that wants to move from point A to B, implement the knowledge and logic to reach the destination.

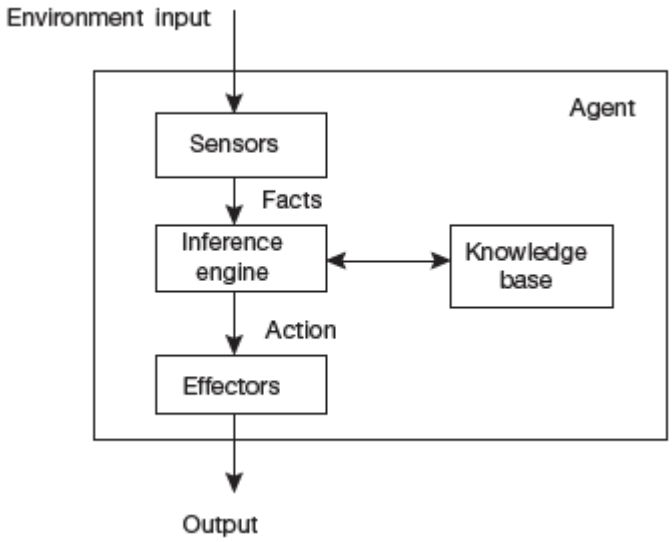


FIGURE 4.7 Adjusting AC temperature

Example: In an automated air conditioner, the inbuilt knowledge stored in the system states that 'It would adjust its temperature according to the weather.' Now, when to adjust and how to adjust is logic. The actual working of adjusting the temperature is at the implementation level of the knowledge-based agent.

4.3.5 Approaches to Designing a Knowledge-Based Agent

There are mainly two approaches to build a knowledge-based agent.

Declarative Approach

In this approach, the knowledge-based agent is created by initializing it with an empty knowledge base and then telling the agent all the sentences with which we want to start with.

These sentences are told to the empty system one by one until the system becomes knowledgeable enough to deal with the environment.

Procedural Approach

In the procedural approach, the desired behaviour is directly encoded in the agent as a program code. Therefore, in this approach, we are actually specifying the desired behaviour of the system through coding in programming languages like LISP, Prolog.

Practically, a **hybrid approach** is used that combines features of both declarative as well as procedural approaches. In the hybrid approach, declarative knowledge is compiled into more efficient procedural code.

4.4 Types of Knowledge

We can express different types of knowledge in a knowledge representation system.

Name	Age	Course
Nrip	19	BCA
Krish	18	BBA
Riya	17	BTech

FIGURE 4.8 A simple relation/ table of students

Simple Relational Knowledge

In this technique, facts about a set of objects are systematically stored using relations (or tables) in database systems (refer **Fig. 4.8**). These relations depict relationship between different entities. However, the main shortcoming of this approach is that there is little opportunity for inference.

Inheritable knowledge Inheritable knowledge is one that stores data using a hierarchy of classes. We start representing using generalized classes and move down to specialized classes in the hierarchy. For example, consider **Fig. 4.9** which shows such a relationship. Note that Zinya and Vidhi both are instances of under-graduate and post-graduate students, respectively. Inheritable knowledge shows the relation between instance and class, and is therefore known as instance relation of IS-A relation. In this diagram, objects and values are represented in boxed nodes.

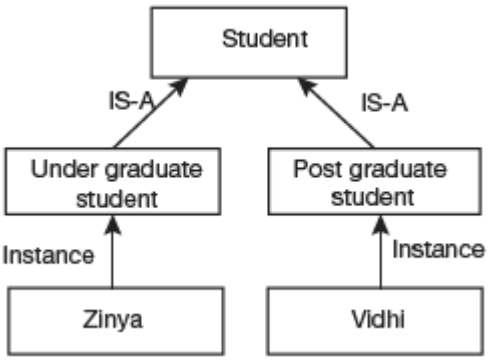


FIGURE 4.9 Inferential Knowledge Representation

Inferential Knowledge

Inferential Knowledge Represents Knowledge in the Form of Formal Logic and Can be Used to Derive More Facts Accurately. For example,

Statement 1: Diya is a student.

Statement 2: All students are bright.

Then, it can be represented as;

Student(Diya)

∀x = Student (x) ———-> Bright (x)

4.5 Techniques of Knowledge Representation in AI

Different techniques of representing knowledge can be given as follows.

4.5.1 Logical Representation

Logical representation is a language with some definite rules. These rules state the proposition without any sort of ambiguity. Rules specify the conclusion based on different conditions. The syntax and semantics of the language guide the process of converting sentences into logics. While syntax of a language is used to decide how legal sentences in logic can be

constructed, semantics, on the other hand, are the rules which helps to interpret (deduce the meaning of) sentences in the logic.

Logical representation can be of two types.

Propositional logic, also known as propositional calculus or statement logic, works in a Boolean, that is, true or false method.

First-order logic (FoL) is an advanced version of propositional logic that represents the objects in quantifiers and predicates.

Language semantics are used to represent this information. The downside of this method is that it is tough to work with as it is highly logical, not very natural and less efficient at times. **Table 4.1** summarizes the advantages and disadvantages of Logic representation techniques

TABLE 4.1 Pros and Cons of Logic Representation

Advantages	Disadvantages
Logic representation is the foundation of any programming language. Logical representation helps to perform logical reasoning.	Logical representations have some restrictions and are challenging to work with. This technique may not be very natural, and inference may not be very efficient.

4.5.2 Semantic Network Representation

Semantic network is used as an alternative of logic representation. It represents knowledge through graphical networks. In a graphical network (refer **Fig. 4.10**), nodes represent objects and arcs describe the relationship between those objects. These objects can be categorized in different forms and can be linked to other objects through two types of relations—IS-A relation (inheritance) and kind-of-relation (instance). **Table 4.2** highlights the advantages and disadvantages of Semantic Network Representation.

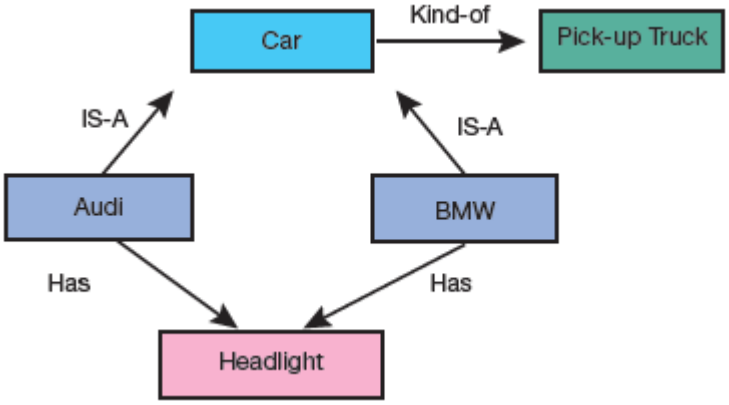


FIGURE 4.10

TABLE 4.2 Pros and Cons of Semantic Networks

Advantages	Disadvantages
• Semantic networks are a natural representation of knowledge. • They convey meaning in a transparent manner. • These networks are simple and easy to understand.	• Semantic networks take more computational time at runtime. • They are inadequate as they do not have any equivalent quantifiers. • These networks are not intelligent and may vary depending on its developer. • Semantic networks do not have any standard definition for the link names. • It is impossible to represent all facts from human's memory into a semantic network.

4.5.3 Frame Representation

A frame is a record-like structure that describes a real world using a collection of attributes and values. A frame also known as slot-filter knowledge representation is derived from semantic networks. These frames at a later stage can be designed as classes and objects using any object-oriented programming language. A single frame is not of much use, so, a collection of inter-connected frames is used. These frames store knowledge about an object or event.

Slot	Filter
Title	Artificial Intelligence
Area	Computer Science
Author	Reema Thareja
Publisher	Pearson
Edition	First
Publication Year	2023
Price	Rs 500
Page Count	700

Frame representation is widely used in various applications including natural language processing and machine visions. Basically, a frame contains slots that, in turn, can store values of any type and size. Every slot has a name and value which we call a facet.

Facets are features of frames which are used to put constraints on the frames. For example, IF-NEEDED facts are called when data of any particular slot is needed. A frame can have any number of slots, a slot may include any number of facets and facets may have multiple values. **Table 4.3** lists the advantages and disadvantages of Frame Representation techniques

Example: Design a frame for a book

TABLE 4.3 Pros and Cons of Frame Representation

Advantages	Disadvantages
• It is easy to program as it groups together the related data. • It is simple to understand and visualize. • The programmer can easily add slots for new attributes and relations. • It is easy to add default data and search for missing values.	• In frame system inference, data processing is not a straight forward task • The inference mechanism finds it difficult to smoothly process data available in frame representation. • It has a very generalized approach.

4.5.4 Production Rules

Production rules system consist of (condition, action) pairs which mean, 'If condition then action'. Using this rule, the agent checks for the condition. If the condition exists, the production rule becomes applicable and the corresponding action is performed.

If we have multiple if conditions or if-else statements in a program, the production rule can also be designed to determine which rule may be applied to a problem. The action part specifies steps that must be performed to solve a problem. This complete process is called a recognize-act cycle.

A production rule system has mainly three parts as sown in **Fig. 4.11**:

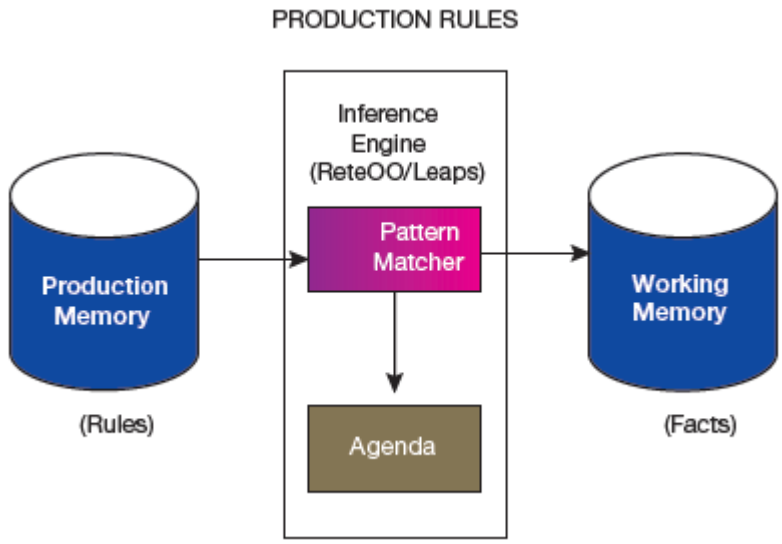


FIGURE 4.11 Components of a Production Rule System

1. The set of production rules
2. Working memory
3. The recognize-act-cycle

The working memory contains the description of the current state of problem-solving.

Rules are used to express knowledge about the working memory. This knowledge may fire other rules.

At times, it happens that multiple production rules are fired together. This situation is called conflict set. In such a situation, the agent needs to select one particular rule. This is often referred to as conflict resolution.

For example, consider the rules given below.

- IF (hungry) THEN action (eat)
- IF (on the bus AND bought ticket AND vacant seat) THEN action (sit down).

TABLE 4.4

Advantages	Disadvantages
• Production rules are expressed in natural language. • They are highly flexible as they can be easily modified.	• It does not have any learning capabilities • It does not store the result of the problem for future uses. • During program execution, multiple rules may be active. • Production rules are inefficient.

4.5.5 Propositional Logic (PL)

Propositional logic is the simplest form of logic in which all the statements are made using propositions. A proposition is a declarative statement which is either true or false but it cannot be both (true as well as false). Knowledge in a proposition is represented in logical as well as mathematical form.

Example:

```
a)  Today is a holiday. (True or False)
b)  The Sun rises from West (False)
c)  1 + 1 = 11 (False)
d)  19 is a prime number.   (True)
```

We can see from the above examples that propositional logic provides better clarity on data and information, especially in an incomplete environment by deeper analysis and inference of the limited information pre-

sented to it. Propositional logic is, therefore, widely adopted in advanced search in AI in solving complex problems.

The following points must be noted.

1. We can express a propositional logic using logical operators.
2. A proposition formula which is always true is called tautology, and is also known as a valid sentence.
3. A proposition formula which is always false is called contradiction.
4. Statements which are questions, commands or opinions are not propositions. For example, 'Which course are you opting for?', 'Read the newspaper', 'I think it's all fiction', are not propositions.
5. An atomic proposition is said to be **satisfiable** if there is an interpretation for which it is true. In **Table 4.6**, P v Q is satisfiable for first three propositional formula (statements).
6. A **contingent** statement is one which is neither a tautology nor a contradiction. For example, a proposition like 'I am not well, so I am taking leave today'. This statement is often neither correct nor incorrect. As we may be having a little dizziness, our main aim behind taking leave is to complete another work on a priority basis.

Proposition means sentences. Propositional logic applies the Boolean logic to convert real-world information into a format that is readable and understandable by the computer.

4.6 Syntax of Propositional Logic

The syntax of propositional logic defines the allowable sentences for the knowledge representation. Two types of propositions include the following:

1. Atomic proposition: Atomic propositions are simple propositions that are made of a single proposition symbol. Such propositions can be either true or false.
   Example:
   1. 2+2 is 4, is an atomic proposition that is true.
   2. The Sun rises in the west is an atomic proposition that is false.

2. Compound proposition: Compound propositions are formed by combining simpler atomic propositions, using parenthesis and logical operators.

   Example:

   1. The Sun rises in the east and sets in the west.
   2. Ruskin Bond prolifically authored inspiring children's books and was awarded the Sahitya Akademi Award to honour his work of literature.

4.7 Logical Connectives in Propositional Logic

Logical connectives are used to form compound propositions by connecting two simpler propositions. Some important and widely used connectives are given as follows:

Negation

A sentence such as ¬ P is called negation of P. A literal can be either positive literal or negative literal.

Conjunction

A sentence which uses 'and' to connect two propositions is said to be connected using a conjunction connective. It is represented using a ∧ connective. Two propositions combined using a conjunction connective can be written as, **P ∧ Q**.

For example, the sentence 'I like to study Mathematics and Computer' can be written as:

P= I like to study Mathematics.

Q= I like to study Computer.

I like to study Mathematics and Computer → P ∧ Q.

Disjunction

A sentence which uses 'or' to connect two propositions is said to be connected using a disjunction connective. It is represented using a ∨ connective. Two propositions combined using a conjunction connective can be written as **P** ∨ **Q**.

For example, the sentence 'I would like to read a book authored by R.K. Narayanan or by Ruskin Bond' can be written as:

P= I would like to read a book authored by R.K. Narayanan

Q= I would like to read a book authored by Ruskin Bond

I would like to read a book authored by R.K. Narayanan or by Ruskin Bond → P ∨ Q.

Implication

A sentence P implies Q can be written as P → Q. This is known as implication or as an if-then rule. For **example:** If you are hungry, then eat some sprouts. Let P= If you are hungry, and Q= Eat some sprouts, can be represented as P → Q

Biconditional

A sentence of the form P ⇔ Q is called a biconditional sentence as P implies Q and Q implies P. For **example:** If you are a consistent learner, then you will never stop learning.

P= You are a consistent learner

Q= You will never stop learning

These two statements can be represented as P ⇔ Q.

Table 4.5 shows different propositional logic connectives.

TABLE 4.5 Propositional logic connectives

Connective symbols	Word	Technical term	Example
∧	AND	Conjunction	A ∧ B
∨	OR	Disjunction	A ∨ B
→	Implies	Implication	A → B
⇔	If and only if	Biconditional	A ⇔ B
¬ or ~	Not	Negation	¬ A or¬ B

4.7.1 Logical Equivalence

Two propositions are said to be logically equivalent if and only if the columns in the truth table are identical to each other. For example, given two propositions P and Q, column for ¬P ∨ Q and P→Q, are identical hence logically equivalent. In addition, note that,

P is valid iif ¬ P is unsatisfiable. And, P is satisfiable iff ¬ P is not valid. We can verify this from **Fig. 4.12**.



FIGURE 4.12 Logical Equivalence

A satisfiable sentence has at least one interpretation under which the sentence can evaluate to be true.

A sentence is valid if it is true in all interpretations. This means that its negation is not satisfiable.

4.7.2 Truth Table of Propositional Connectives

The following truth table (**Table 4.6**) shows propositional logic, showing all the possible combinations with logical connectives in a tabular format.

Note that the truth table follows Boolean logic and propositional calculus.

TABLE 4.6 Propositional Connectives

Negation

P	~P
TRUE	FALSE
FALSE	TRUE

Conjunction

P	Q	P ∧ Q
FALSE	FALSE	FALSE
FALSE	TRUE	FALSE
TRUE	FALSE	FALSE
TRUE	TRUE	TRUE

Disjunction

P	Q	P ∨ Q
FALSE	FALSE	FALSE
FALSE	TRUE	TRUE
TRUE	FALSE	TRUE
TRUE	TRUE	TRUE

Implication

P	Q	~P	P ∧ Q	P → Q can be understood as, {~P∨(P ∧ Q)}
FALSE	FALSE	TRUE	FALSE	TRUE
FALSE	TRUE	TRUE	FALSE	TRUE
TRUE	FALSE	FALSE	FALSE	FALSE
TRUE	TRUE	FALSE	TRUE	TRUE

Logical Equivalence

P	Q	~P	~P ∨ Q	P → Q
FALSE	FALSE	TRUE	TRUE	TRUE
FALSE	TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	FALSE	FALSE
TRUE	TRUE	FALSE	TRUE	TRUE

Bi-directional implication

P	Q	P ⇔ Q
FALSE	FALSE	TRUE
FALSE	TRUE	FALSE
TRUE	FALSE	FALSE
TRUE	TRUE	TRUE

4.7.3 Precedence of Connectives

We have an operator precedence chart in any programming language. Similarly, there is a precedence order for propositional connectors (shown in **Table 4.7**). This order is strictly followed while evaluating a propositional problem.

TABLE 4.7 Propositional Connectors

Precedence	Operators
First precedence	Parenthesis
Second precedence	Negation
Third precedence	Conjunction (AND)
Fourth precedence	Disjunction (OR)
Fifth precedence	Implication
Sixth precedence	Biconditional

4.7.4 Properties of Operators

Commutativity

$P \wedge Q = Q \wedge P$, or $P \vee Q = Q \vee P$.

Associativity

$(P \wedge Q) \wedge R = P \wedge (Q \wedge R)$, or $(P \vee Q) \vee R = P \vee (Q \vee R)$

Identity Element

$P \wedge True = P$ or $P \vee True = True$

Distributive

$P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R)$ or $P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$.

DE Morgan's Law

$\neg (P \wedge Q) = (\neg P) \vee (\neg Q)$ or $\neg (P \vee Q) = (\neg P) \wedge (\neg Q)$.

Double-Negation Elimination

$\neg (\neg P) = P$.

4.7.5 Limitations of Propositional Logic

Not all relationships can be represented using propositional logic. For example, relations like ALL, some or none cannot be represented with propositional logic. For example,

All students are studious

Some students are outstanding.

Statements which cannot be answered absolutely are called open statements. It is not possible to describe such statements in terms of their properties or logical relationships using propositional logic. For example, 'It is so easy' is an open statement as it may be simple for some but difficult for others.

4.8 Inference Rules

We know that AI is being used to create intelligent computers that can generate create new logic from old ones or by evidence. This process of generating conclusions from evidence and facts is known as inference.

Inference rules are the templates for generating valid arguments that are used to derive proofs to obtain conclusions that finally lead to the desired goal. Some important terminology frequently used in inference rules include the following.

Implication

Logical connectives are represented as $P \rightarrow Q$. It is a Boolean expression.

Converse

The converse of implication, which means that the right-hand side proposition goes to the left-hand side and vice-versa can be written as Q → P.

Contrapositive

The negation of converse represented as ¬ Q → ¬ P, is known as contrapositive.

Inverse

The negation of implication is called inverse. It can be represented as ¬ P → ¬ Q.

The truth table given below (**Table 4.8**) illustrates these relationships. We can see that P → Q is equivalent to ¬ Q → ¬ P, and Q→ P is equivalent to ¬ P → ¬ Q.

TABLE 4.8 Inverse Relations

P	Q	P→Q	Q→P	¬Q→¬P	¬P→¬Q
T	T	T	T	T	T
T	F	F	T	F	T
F	T	T	F	T	F
F	F	T	T	T	T

4.8.1 Types of Inference Rules

Modus Ponens: The Modus Ponens rule is an important inference rule that states if P is True and P → Q is True, then we can infer that Q will also be true. This can be represented as,

$$\text{Notation for modus ponens: } \frac{P \rightarrow Q, P}{\therefore Q}$$

Example:

Statement-1: 'If I am studying thoroughly then I have a test next day.' ==> P→ Q

Statement-2: ' I am studying thoroughly ' ==> P

Conclusion: ' I have a test next day.' ==> Q.

In the last row, P → Q is true, P is true and therefore, Q is also true (**Fig. 4.13**).

P	Q	~P	P → Q (~P ∨ Q)
FALSE	FALSE	TRUE	TRUE
FALSE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	FALSE
TRUE	TRUE	FALSE	TRUE

FIGURE 4.13 Truth Table for Modus Ponens

Hence, we can say that, if P→ Q is true and P is true then Q will also be true. Figure proves this inference rule with the help of a truth table.

Modus Tollens: The Modus Tollens rule states that if P→ Q is True and ¬ Q is True, then ¬ P will also true. It can be written as,

$$\text{Notation for Modus Tollens: } \frac{P \rightarrow Q, \sim Q}{\sim P}$$

P	Q	~P	~Q	P → Q (~P ∨ Q)
FALSE	FALSE	TRUE	TRUE	TRUE
FALSE	TRUE	TRUE	FALSE	TRUE
TRUE	FALSE	FALSE	TRUE	FALSE
TRUE	TRUE	FALSE	FALSE	TRUE

In the last row, P→ Q is true, ~Q is true indicates that ~P is also ture.

Statement-1: 'If I am studying thoroughly then I have a test next day.' ==> P→ Q

Statement-2: 'I am not having a test tomorrow' ==> ~Q

Statement-3: We can conclude from the above two statements that ' I am not studying thoroughly' ==> ~P

Hypothetical Syllogism: The Hypothetical Syllogism rule states that P→R is True iff P→Q is true, and Q→R is true.

Example:

Statement-1: If you study hard then you will get good knowledge. **P→Q**

Statement-2: If you have good knowledge then you can get recruited easily. **Q→R**

Conclusion: If you study hard then you can get recruited easily. **P→R**

This rule can be proved using a truth table (**Table 4.9**) as shown below.

TABLE 4.9 Truth Table for Hypothetical Syllogism

P	~P	Q	~Q	R	~R	P → Q (~P ∨ Q)	Q → R (~Q ∨ R)	P → R (~P ∨ R)
FALSE	TRUE	FALSE	TRUE	FALSE	TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	TRUE	TRUE	FALSE	TRUE	TRUE	TRUE
FALSE	TRUE	TRUE	FALSE	FALSE	TRUE	TRUE	FALSE	TRUE
FALSE	TRUE	TRUE	FALSE	TRUE	FALSE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE	FALSE	TRUE	FALSE	TRUE	FALSE
TRUE	FALSE	FALSE	TRUE	TRUE	FALSE	FALSE	TRUE	TRUE
TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	TRUE	FALSE	FALSE
TRUE	FALSE	TRUE	FALSE	TRUE	FALSE	TRUE	TRUE	TRUE

Notation of disjunctive syllogism: $\frac{P \vee Q, \neg P}{Q}$

Disjunctive Syllogism: The Disjunctive syllogism rule states that if P∨Q is True, and ¬P is True, then Q will be true. It can be expressed as,

Example:

Statement 1: Today I will study AI or Java. ==>P∨Q

Statement 2: Today I will not study AI. ==> ¬P

Conclusion: I will Study Java. ==> Q

TABLE 4.10 Truth Table for Disjunctive Syllogism

P	~P	Q	P ∨ Q
FALSE	TRUE	FALSE	FALSE
FALSE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE
TRUE	FALSE	TRUE	TRUE

Statement-1: Today is Sunday or Monday.

Statement-2: Today is not Sunday.

Conclusion: Today is Monday.

This rule is also proved using truth table given (**Table 4.10**).

Addition: The Addition rule is a common inference rule that states, if P is true, then P∨Q will be true.

$$\text{Notation of addition: } \frac{P}{P \vee Q}$$

Example:

Statement: I would like to study Mathematics. ==> P

Statement-2: I would like to study Artificial Intelligence. ==> Q

Conclusion: I would like to study Mathematics or Artificial Intelligence. ==> (P∨Q)

Simplification: The simplification rule state that if **P ∧ Q** is true, then both **Q as well as P** will also be true. It can be represented as:

$$\text{Notation of simplification rule: } \frac{P \wedge Q}{Q} \text{ or } \frac{P \wedge Q}{P}$$

Resolution: The Resolution rule state that if P ∨ Q and ¬ P ∧ R is true, then Q ∨ R will be true. It can be represented as

$$\text{Notation of resolution: } \frac{P \vee Q, \neg Q \wedge R}{P \vee R}$$

Proof by Truth table (**Table 4.11**):

TABLE 4.11 Truth Table for Resolution

P	Q	~Q	R	P ∨ Q	~Q ∨ R	P ∨ R
FALSE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE
FALSE	FALSE	TRUE	TRUE	FALSE	TRUE	TRUE
FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE
FALSE	TRUE	FALSE	TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	TRUE	FALSE	TRUE	TRUE	TRUE
TRUE	FALSE	TRUE	TRUE	TRUE	TRUE	TRUE
TRUE	TRUE	FALSE	FALSE	TRUE	FALSE	TRUE
TRUE	TRUE	FALSE	TRUE	TRUE	TRUE	TRUE

4.8.2 First-Order Logic

In the previous section, we learnt how propositional logic can be used to represent statements. Although it is a very powerful technique, it suffers from a major shortcoming. It can represent the facts, which are either true or false. Complex sentences or natural language statements cannot be represented using propositional logic. Thus, it has a very limited expressive power. For example, consider the statements given below. These statements cannot be expressed in propositional logic.

'Some boys are hard-working', or

‘Sania likes badminton.’

We need a more powerful logic, such as first-order logic (also known as predicate logic) to represent such statements. First-order logic (FOL) is another way of knowledge representation that can be thought of as an extension to propositional logic. It can easily represent natural language statements in a concise way.

Like natural language, first-order logic assumes that the world contains facts (like propositional logic) as well as the following things in the world:

Objects

People, alphabets, numbers, colours, theories, squares, pits, wumpus, etc.

Relations

A relation can be a simple unary relation such as red, round, is adjacent, or an n-ary relation like the sister of, brother of, has colour, comes between.

Function

Mother of, best friend, second match of, end of,……

Moreover, like natural language, first-order logic also has two main parts —syntax and semantics.

4.8.2.1 Syntax of First-Order Logic

The syntax of FOL determines the set of symbols that can be used in a logical expression in first-order logic. These symbols, which are the basic elements of a FOL sentence, comprise the following.

Atomic Sentences

Atomic sentences are the most basic sentences of FOL. They are formed using a predicate followed by a parenthesis with a sequence of terms. The general syntax of writing an atomic sentence in FOL can be given as,

Figure 4.14

Predicate (term1, term2,……, term n).

For example,

Ria and Krisha are sisters can be expressed in FOL as Sisters(Ria, Krisha).

Bruno is a dog. => dog(Bruno).

Constant	1, -10, F, Myra, Delhi, cow,....
Variables	x, y, z, a, b,....
Predicates	Father, Enjoys, Reading,....
Function	sqrt, LeftLegOf,....
Connectives	$\wedge, \vee, \neg, \Rightarrow, \Leftrightarrow$
Equality	==
Quantifier	$\forall, \exists$

FIGURE 4.14 Basic elements of a FOL sentence

Complex Sentences

Complex sentences, as the name suggests, are formed by combining atomic sentences using connectives. A complex sentence in FOL has two main parts—subject and predicate.

Subject is the main part of the statement. It specifies what (or whom) the sentence is about.

Predicate is a relation which binds two atoms together in a statement. It tells something about the subject.

Consider the statement: ‘Ahana is a good student.’. It consists of two parts —the first part ‘Ahana’ is the subject and the second part ‘is a good student’ is the predicate.

Quantifiers

A quantifier is an element that generates quantification. Quantification specifies the quantity of specimen in the universe of discourse. In simple words, it identifies the range and scope of the variable in the logical expression. There are two types of quantifiers: universal quantifier, (for all, everyone, everything) and existential quantifier, (for some, at least one).

Universal quantifier: Universal quantifier is a symbol of logical representation written as ∀, which specifies that the statement (within its range) is true for everything or every instance of a particular thing. That is, if x is a variable, then ∀x is read as- For all x, For each x, For every x.

In universal quantifier, implication (→) is used as the main connective.

For **example:** Every student who works hard succeed can be represented as,

∀x student(x) → work_hard (x, succeed).

Existential quantifier: Existential quantifiers, denoted by the logical operator ∃, are used to express that the statement within its scope is true for at least one instance of something. In simple terms, if x is a variable, then ∃x or ∃(x) will be read as,

- There exists a ‘x.’
- For some ‘x.’
- For at least one ‘x.’

In existential quantifier, AND or Conjunction symbol (∧) is used as the main connective.

For example, the statement, ‘Some students are brilliant’, can be written as,

∃x: students(x) ∧ brilliant(x)

4.8.2.2 Properties of Quantifiers

Before writing predicate rules, always keep the following properties of quantifiers in your mind.

1. In universal quantifier, ∀x∀y is similar to ∀y∀x.
2. In existential quantifier, ∃x∃y is similar to ∃y∃x.
3. However, ∃x∀y is not similar to ∀y∃x.

Let us look at some more examples of FOL using quantifiers.

1. Every child cries.
   Here, the predicate is 'cries(child).' Since every child cries, it can be represented using the universal quantifier by writing,
   ∀x child(x) → cry(x).
2. Every student writes exams.
   Here, the predicate is 'write(x, y),' where x = student, and y= exam.
   Again, we will use the universal quantifier to represent this fact.
   ∀x student (x) → writes (x, exam).
3. Some students have participated in the Olympiad.
   Here, the predicate is 'participate(x, y)' where x= students, and y= Olympiad. Since only few and not every student has participated, we will represent this statement using existential quantifier by writing,
   ∃x students (x) → participate(x, Olympiad).
4. Not every student likes both Mathematics and Computers.
   Here, the predicate is 'like(x, y),' where x= student, and y= subject (Mathematics, Computer).
   Note that here, we have used every but with negation. Therefore, this statement can be written as,
   ¬∀ (x) [student(x) → like(x, Mathematics) ∧ like(x, Computers)].
5. Only one student participated in the Olympiad.
   Here, the predicate is 'participated (x, y),' where x= student, and y= Olympiad.
   Now that only one student participated in Olympiad, we will use existential quantifier and write,
   ∃(x) [student(x) → participated (x, Olympiad) ∧ ∀ (y) [¬(x==y) ∧ student(y) → ¬participated (x, Olympiad)].

4.8.2.3 Free and Bound Variables

The quantifiers usually interact with variables. These variables are of two types.

Free variable: A variable in a formula is said to be a free variable if it occurs outside the scope of the quantifier. For example,

∀x ∃(y)[P (x, y, z)]

Here, neither existential nor universal quantifier is applicable on variable z. hence, it is a free variable.

Bound variable: On the contrary, a bound variable in a formula occurs within the scope of the quantifier. For example,

∀x [A (x) B(y)]

Here, x and y are the bound variables.

4.8.2.4 Inference in First-Order Logic

Inference in FOL is used to deduce new facts or sentences from existing ones. To understand the FOL inference rule, let us learn about some basic terminologies that will be frequently used in FOL.

Substitution

Substitution is a fundamental operation that can be easily performed on terms and formulas to derive a new inference. If we write **F[a/x]**, then it means replace constant value '**a**' with variable '**x**'.

The substitution is complex in the presence of quantifiers in FOL.

Equality

FOL uses use equality symbols to specify that the two terms refer to the same object. For example,

Teacher (AI) = Raman

The above sentence indicates that the object referred by the Teacher (AI) is similar to the object referred to by Raman. The equality symbol can also be used with negation to represent those two terms are not the same objects. For example, we can write ¬(x=y) to represent x ≠ y.

4.8.2.5 FOL Inference Rules for Quantifier

Like propositional logic, inference rules in FOL also follows some basic rules such as universal generalization, universal instantiation, existential instantiation and existential introduction.

Universal Generalization

Universal generalization states that for any arbitrary element c in the universe of discourse, if P(c) is true, then we can conclude that

$$\frac{P(c)}{\forall x P(x)}$$

This formula indicates that every element has a similar property. Hence, x must not appear as a free variable.

For example, if P(c): 'A nibble contains 4 bits'**.** We can use this information to give a universal generalization as,

∀ x P(x) 'All nibbles contain 4 bits'

Universal Instantiation

Universal instantiation, also known as universal elimination or UI, is an inference rule that can be applied multiple times to add new sentences in the knowledge base. The new or the updated knowledge base is logically equivalent to the previous one.

According to UI, we can infer any sentence P(c) by substituting a ground term, c (a constant within domain x) for the variable**.** Therefore,

∀ x P(x) for any object in the universe of discourse, we can write

$$\frac{\forall x P(x)}{P(c)}$$

Example: IF 'Every person reads books' => ∀x P(x), we can infer that

'Navya reads books ' => P(c).

Example: 'All students who scored 90+ are outstanding.' This fact can be added in the knowledge base by writing,

∀x student(x) ∧ scored_90+ (x) → Outstanding (x)

Now, this information can be used to infer any of the following statements using UI.

- Student(Manya) ∧ Scored_90+ (Manya) → Outstanding (Manya),
- Student(Shrey) ∧ Scored_90+ (Shrey) → Outstanding (Shrey),
- Student(Sister(Prince)) ∧ Scored_90+ (Sister(Prince)) → Outstanding (Sister(Prince))

Existential Instantiation

Existential instantiation or existential elimination is an inference rule in FOL that can be applied only once to replace the existential sentence.

The new knowledge base is not logically equivalent to the previous one, but it will be satisfiable if the older version of the knowledge base was. The existential instantiation inference rule states that P(c) can be inferred from the formula given in the form of ∃x P(x) for a new constant symbol c if and only if c is a new term for which P(c) is true. It can be represented as:

$$\frac{\exists x P(x)}{P(c)}$$

For example, from the given sentence, **∃x Winner(x) ∧ CashPrize(x, 1000)**, we can infer: Winner(**K) ∧ CashPrize(K, 1000)**, only if K does not

appear in the knowledge base. Here, K is a constant symbol and is known as **Skolem constant**.

Existential Introduction

Existential introduction, also known as existential generalization, is an inference rule in FOL which states that if there is some element c in the universe of discourse having a property P, then we can infer that there exists something in the universe which has the property P. It can be represented as:

$$\frac{P(c)}{\exists x P(x)}$$

The existential instantiation is a special case of **Skolemization process**.

Example: If we say that Aman won a cash prize of Rs 1000, then we can infer that there is someone who won the cash prize of Rs 1000.

4.8.3 Generalized Modus Ponens Rule

In FOL, we have a single inference rule known as generalized modus ponens that is a lifted version of modus ponens. The generalized modus ponens states that if P implies Q and P is asserted to be true, then Q must be true.

In the context of atomic sentences pi, pi', q, Modus Ponens states that if there is a substitution $\boldsymbol{\theta}$ such that SUBST ($\boldsymbol{\theta}$, pi',) = SUBST($\boldsymbol{\theta}$, pi), then it can be represented as:

$$\frac{p1', p2', \ldots, pn', (p1 \wedge p2 \wedge \ldots \wedge pn \Rightarrow q)}{SUBST(\theta, q)}$$

For example, let us use this rule to find some x such that x is student, and x is scoring above 90 and infer that x is outstanding.

p1' is Student(Myra) p1 is Student(x)

p2' is Scored_90+(y) p2 is Scored_90+(x)

$\boldsymbol{\theta}$ is {x/Myra, y/Myra} q is Outstanding(x)

SUBST($\boldsymbol{\theta}$,q).

4.8.4 Unification in FOL

The unification process is used to make two different logical atomic expressions (or literals) identical by finding a substitution.

Let $\boldsymbol{\Psi_1}$ and $\boldsymbol{\Psi_2}$ be two atomic sentences and $\boldsymbol{\sigma}$ be a unifier such that $\boldsymbol{\Psi_1\sigma} = \boldsymbol{\Psi_2\sigma}$, then it can be expressed as UNIFY($\boldsymbol{\Psi_1}$, $\boldsymbol{\Psi_2}$).

For example, to unify {Student(x), Student (Jerry)}, we will write

$$\Psi_1 = \text{Student (x)}, \ \Psi_2 = \text{Student (Jerry)},$$

Now, using the substitution process, we will substitute Jerry for x. Therefore,

$\boldsymbol{\theta}$ = {Jerry/x} is a unifier for these atoms. Using the substitution process, both expressions are found to be identical.

The substitution variables are called most general unifier or MGU.

4.8.5 Unification Algorithm

Unification is a very important part of first-order inference algorithms. The UNIFY algorithm unifies two atomic sentences and returns a unifier for those sentences, if one exists. However, if the two sentences are not identical, the UNIFY algorithm returns failure, indicating that the two sentences do not match.

Example: Given two expressions, P(x, y), and P(a, f(z)), find out if they are identical to each other.

Let us perform the substitution process. 579

P(x, y)	(i)
P(a, f(z))	(ii)

In equation (i), substitute x with a, and y with f(z). This can be written in FOL substitution as a/x and f(z)/y.

With both the substitutions, equation (i) will be identical to the second expression- P(a, f(z).

Hence, the two expressions are identical when the substitution set is: [a/x, f(z)/y].

Although unification seems simple and straight forward, we need to satisfy some basic conditions for this process, described below.

1. Predicate symbol must be the same. Expressions with different predicates cannot be unified.
2. Number of arguments in both the expressions must be identical.
3. Unification works only when different variables are used. Thus, the unification process will fail if two similar variables are present in the same expression.

Figure 4.15 gives the algorithm for unification.

```
Algorithm: Unify(Ψ1, Ψ2)
Step 1: If Ψ1 or Ψ2 is a variable or constant, then:
      a) If Ψ1 or Ψ2 are identical, then return NIL.
      b) Else if Ψ1is a variable,
            a. then if Ψ1 occurs in Ψ2, then return FAILURE
            b. Else return {(Ψ2/ Ψ1)}.
      c) Else if Ψ2 is a variable,
            a. If Ψ2 occurs in Ψ1 then return FAILURE,
            b. Else return {(Ψ1/ Ψ2)}.
      d) Else return FAILURE.
Step 2: If the initial Predicate symbol in Ψ1 and Ψ2 are not same,
then return FAILURE.
Step 3: IF Ψ1 and Ψ2 have a different number of arguments, then
return FAILURE.
Step 4: Set Substitution set(SUBST) to NIL.
Step 5: For i=1 to the number of elements in Ψ1.
      a) Call Unify function with the ith element of Ψ1 and ith
element of Ψ2, and put the result into S.
      b) If S = failure then returns Failure
      c) If S ≠ NIL then do,
            a. Apply S to the remainder of both L1 and L2.
            b. SUBST= APPENDS, SUBST).
Step 6: Return SUBST.
```

FIGURE 4.15 Algorithm for unification

Note that in the algorithm, we have initially set the substitution set to be empty. Then, with each pass, every atomic sentence is unified, if the following conditions are applicable.

1. There is an identical expression match.
2. If one expression is a variable v_i, and the other is a term t_i which does not contain variable v_i, then,
    1. Substitute t_i / v_i in the existing substitutions.
    2. Add t_i /v_i to the substitution setlist.
    3. If both the expressions are functions, then check if the function name and the number of arguments are the same in both the expressions.

Example: Find the MGU of {p(f(a), g(Y)) and p(X, X)}

Let $\boldsymbol{\Psi_1}$ = p(f(a), g(Y)) and $\boldsymbol{\Psi_2}$ = p(X, X)

SUBST $\boldsymbol{\theta}$= {f(a) / X}

We get $\boldsymbol{\Psi_1}$ = p(f(a), g(Y)) and $\boldsymbol{\Psi_2}$ = p(f(a), f(a)).

Since f(a) is not same as g(Y), unification failed.

Example: Find the MGU of {p(b, X, f(g(Z))) and p(Z, f(Y), f(Y))}

Let Ψ_1 = p(b, X, f(g(Z))) and Ψ_2 = p(Z, f(Y), f(Y))

SUBST θ = {b/Z}

We get {p(b, X, f(g(b))) and p(b, f(Y), f(Y))}.

Now, SUBST θ = {f(Y) /X}

We get, {p(b, f(Y), f(g(b))) and p(b, f(Y), f(Y))}

SUBST θ = {g(b) /Y}

We finally get,

{p(b, f(g(b)), f(g(b)) and p(b, f(g(b)), f(g(b))}

Hence, unification process is successfully done and the unifier = { b/Z, f(Y) /X , g(b) /Y}.

Example: Find the MGU of {p (X, X), and p (Z, f(Z))}

Let, Ψ_1 = {p (X, X), and Ψ_2 = p (Z, f(Z))

SUBST θ= {X/Z}

We get, {p (Z, Z) and p (Z, f(Z))}

Since the two expressions are not identical, unification for these expressions is not possible.

Example: Find the MGU of UNIFY(prime (11), prime(y))

Let, Ψ_1 = {prime(11) , and Ψ_2 = prime(y)}

SUBST θ= {11/y}

We get, prime(11) and prime(11)}

Hence, successfully unified and Unifier is {11/y}.

Example: Find the MGU of Q(a, g(x, a), f(y)), Q(a, g(f(b), a), x)}.

Let Ψ_1 = Q(a, g(x, a), f(y)), and Ψ_2 = Q(a, g(f(b), a), x)

SUBST θ= {f(b)/x}

We get Q(a, g(f(b), a), f(y)) and Q(a, g(f(b), a), f(b))}

Now, SUBST θ= {b/y}

Finally, Q(a, g(f(b), a), f(b)) and Q(a, g(f(b), a), f(b))

Hence, successfully unified and the Unifier is [a/a, f(b)/x, b/y].

Example: UNIFY(knows(Richard, x), knows(Richard, John))

Let Ψ_1 = knows(Richard, x), and Ψ_2 = knows(Richard, John)

SUBST θ= {John/x}

We get, knows(Richard, John) and knows(Richard, John).

Hence, successfully unified with unifier as {John/x}.

4.8.6 Resolution

Resolution is a theorem proving technique invented by mathematician John Alan Robinson in 1965. The technique stresses on building refutation proofs, that is, proofs by contradictions. Resolution is usually used when there are multiple statements and we have to prove a conclusion.

Unification is an important part in proving by resolutions. Resolution derives a single inference rule which can efficiently operate on the conjunctive normal form (CNF) or clausal form**.** Here,

Clause is a disjunction of literals or an atomic sentence.

Conjunctive normal form is a sentence represented as a conjunction of clauses.

Resolution technique is used in both propositional as well as first-order predicate logic in different ways for proving the satisfiability of a sentence. For this, resolution applies **Proof by Refutation** in which the knowledge base and the negated goal is used to obtain a null clause. The null clause indicates contradiction. While the knowledge base itself is consistent, the contradiction is of course introduced by a negated goal thereby indicating that the original goal is true. Hence, the name, proof by refutation.

4.8.7 The Resolution Inference Rule

A resolution can resolve two clauses if they contain complementary literals. The underlying assumption here is that the literals are standardized and share no variables with each other.

$$\frac{l_1 V \ldots V l_k, \quad m_1 v \ldots V m_n}{SUBST(\sigma, l_1 V \ldots V l_{i-1}\ V l_{i+1}\ V \ldots V\ l_k\ V m_1 V \ldots V m_{j-1}\ V m_{j+1} V \ldots V m_n)}$$

where li and mj are complementary literals.

This rule is also called the binary resolution rule because it only resolves exactly two literals. In simple terms, to show that a formula X is valid, we show that ¬X is unsatisfiable. That is, given a consistent set of axioms, A and a goal B, to show A |= B, we show that (A ∧ ¬B) is unsatisfiable.

Example: Resolve the two clauses given below:

[Animal (g(x) V Loves (f(x), x)] and [¬ Loves(a, b) V ¬Kills(a, b)]

Here, we have two complimentary literals: Loves (f(x), x) and ¬ Loves (a, b)

We can perform unification of these literals with unifier ***θ***= [a/f(x), and b/x] to generate a resolvent clause:

$$[\text{Animal}(g(x)) \vee \neg \text{Kills}(f(x), x)]$$

4.8.8 The Resolution Process

Given below are steps that need to be performed in resolution process.

Step 1: Convert facts into first-order logic statements.

Step 2: Convert FOL statements into CNF.

Step 3: Negate the statement which needs to be proved as we will prove by contradiction

Step 4: Draw the resolution graph (unification).

Example:

- Mannat likes all kind of food.
- Mango and LadyFinger are food
- Anything anyone eats and not killed is food.
- Krish eats almonds and still alive
- Reyansh eats everything that Krish eats.

Now, prove by resolution that Mannat likes almonds

Step-1: Convert these facts into FOL statements.

$\forall x$: food(x) $\rightarrow$ likes(Mannat,x)
food(mango) $\wedge$ food(LadyFinger)
$\forall x \forall y$: eats(x,y) $\wedge \neg$ killed(x) $\rightarrow$ food(y)
eats(Krish, almonds) $\wedge$ alive(Krish)
$\forall x$ eats(Krish,x) $\rightarrow$ eats(Reyansh,x)
$\forall x$: $\neg$ killed(x) $\rightarrow$ alive(x)
$\forall x$: alive(x) $\neg$ killed(x)
likes(Mannat, almonds)

↕ Newly added predicates as they will be required for the purpose

Step-2: Conversion of FOL into CNF as CNF makes easier for resolution proofs.

For this, first eliminate all implication. This is done by replacing → rewriting the expression. This means, replace

a → b with ¬ a V b

If there is a biconditional implication as, a ⟷ b, then write **a → b** and **b → a**

Therefore, we now get,

∀x ¬ food(x) V likes(Mannat, x)

food(mango) ∧ food(LadyFinger)

∀x ∀y ¬ [eats(x, y) ∧ ¬ killed(x)] V food(y)

eats(Krish, almomds) ∧ alive(Krish)

∀x ¬ eats(Krish,x) V eats(Reyansh,x)

∀x¬ [¬killed(x)] V alive(x)

∀x ¬ alive(x) V ¬ killed(x)

likes(Mannat, almonds)

Now, move negation (¬) inwards and rewrite the expressions. For example,

¬(∀x p) will be written as ∃x ¬p

¬(∃x p) will be written as ∀x ¬p

¬ (a V b) will be written as ¬a ∧ ¬b

¬(a ∧ b) will be written as ¬a V ¬b

¬¬a will be written as a

Therefore, we get,

∀x ¬ food(x) V likes(Mannat, x)

food(mango) ∧ food(LadyFinger)

∀x ∀y ¬eats(x, y) V killed(x) V food(y)

eats(Krish, almomds) ∧ alive(Krish)

∀x ¬ eats(Krish,x) V eats(Reyansh,x)

∀x killed(x) V alive(x)

∀x ¬ alive(x) V ¬ killed(x)

likes(Mannat, almonds)

Rename variables or standardize variables to ensure that every statement has a unique variable name. We should not repeat 'x' or any other variable in any subsequent statement. This is important to make unification step easier and unambiguous. Therefore, we will write,

∀x ¬ food(x) V likes(Mannat, x)

food(mango) ∧ food(LadyFinger)

∀u ∀v ¬eats(u,v) V killed(u) V food(v)

eats(Krish, almomds) ∧ alive(Krish)

∀x ¬ eats(Krish,w) V eats(Reyansh,w)

∀x killed(z) V alive(z)

∀x ¬ alive(g) V ¬ killed(g)

likes(Mannat, almonds)

Eliminate existential instantiation quantifier **(∃)** by replacing it with skolem constant. This process is known as **Skolemization**. For example,

∃y Intelligent(y) is now written as Intelligent(C_1)

Here, C_1 is the skolem constant.

In our example problem, since there is no existential quantifier, all the statements will remain same in this step.

Drop Universal quantifiers (∀) and rewrite the statements.

¬ food(x) V likes(Mannat, x)

food(mango) ∧ food(LadyFinger)

¬eats(u,v) V killed(u) V food(v)

eats(Krish, almomds) ∧ alive(Krish)

¬ eats(Krish,w) V eats(Reyansh,w)

killed(z) V alive(z)

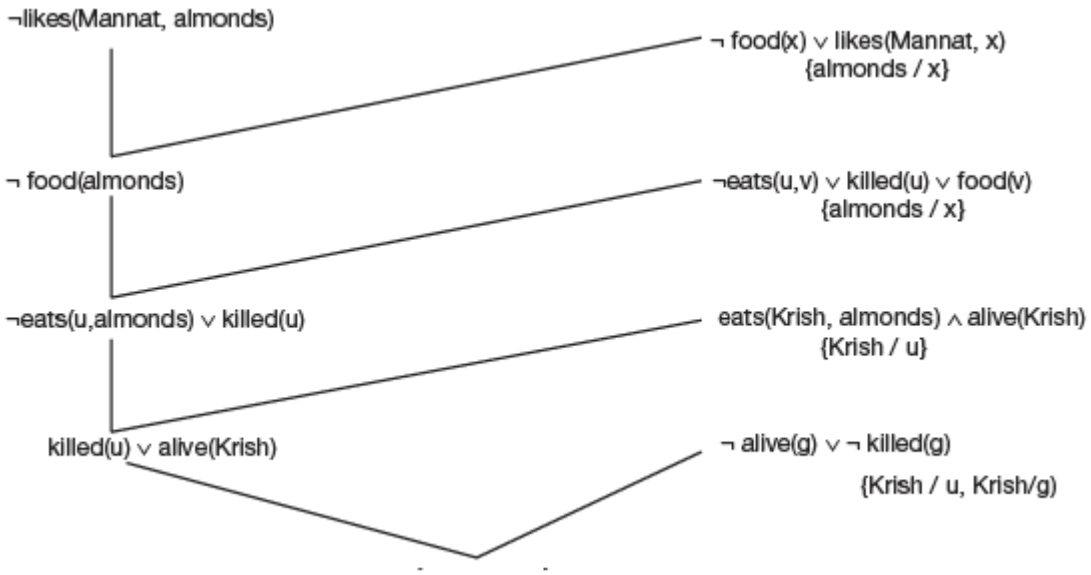
¬ alive(g) V ¬ killed(g)

likes(Mannat, almonds)

Distribute conjunction ∧ over disjunction ¬. This step will not make any change in this problem.

Step-3: Negate the statement to be proved. So we will write ¬likes(Mannat, almonds)

Step-4: Draw Resolution graph using substitution.



To construct the resolution graph, note that on the LHS we would write the statement that is the negation of what has to be proved.

On the RHS, we will write predicate that has the same predicate as that on the LHS. Since we have likes on the LHS, we choose to write a predicate 'likes' on the RHS. Moreover, we will substitute variable x with almonds. Now, eliminate likes and not likes so that ¬food(almonds) is left.

Repeat the process. Find a clause having the predicate food to eliminate the common ones. Going like this, we will come to a point when no predicates would be left. An empty set indicates the negation of the conclusion has been proved as a complete contradiction with the given set of statements.

4.8.9 Explanation of Resolution Graph

In the first step of a resolution graph, ¬ likes(Mannat, almonds)and **likes**(Mannat, **x**) get resolved or cancelled by substitution of **{almonds/x}**. We are left with ¬ **food(almonds)**.

In the second step of the resolution graph, ¬ **food(almonds)** and **food(v)** get resolved or cancelled by substitution of **{almonds /v}**. We are left with ¬ **eats(u, almonds) V killed(y)**.

In the third step of the resolution graph, ¬ **eats(u, almonds)** and **eats(Krish, almonds)** gets resolved by substitution **{Krish /u}**. We are left with

killed(Krish) and alive **(Krish)**.

In the fourth step of the resolution graph, both the predicates get resolved after substituting Krish for the variables. Now, no predicates are left to be resolved.

4.9 Forward Chaining and Backward Chaining in AI

We know that inference engine is the component of the intelligent system that applies logical rules to the knowledge base to infer new information from the known facts. The first inference engine was used in an expert system and it mainly uses two modes—**forward chaining and backward chaining**.

Artificial intelligence (AI) system uses both forward as well as backward chaining to perform tasks like planning, classification, reasoning, etc. The importance of these two techniques is immense and hence there is a need to understand their basics.

4.9.1 Horn Clause and Definite Clause

Horn clause and definite clause are the forms of sentences, which facilitates knowledge base to use a more restricted and efficient inference algorithm. These logical inference algorithms apply forward and backward chaining approaches. But before applying these approaches, the knowledge base must be in the form of the **first-order definite clause**.

Definite Clause

A clause which is a disjunction of literals with exactly one positive literal is known as a definite clause or strict horn clause.

Horn Clause

A clause which is a disjunction of literals with at most one positive literal is known as horn clause. Hence, all the definite clauses are horn clauses.

Example: (¬ p V ¬ q V k)**.** It has only one positive literal k.

It is equivalent to $p \wedge q \rightarrow k$.

4.9.2 Forward Chaining

Forward chaining also known as a forward deduction or forward reasoning and starts with atomic sentences in the knowledge base and applies inference rules (Modus Ponens) in the forward direction to extract more data until a goal is reached. In simpler terms, forward-chaining algorithm starts from known facts, applies rules to deduce new facts and then adds those new facts to the known facts. This process is repeated until the problem is solved (or the goal state is attained).

Forward chaining is a popular implementation strategy used for improving and developing expert systems and modelling the human brain in AI.

Forward chaining approach is also said to be data-driven approach as we reach the goal using available data. It works using bottom-up approach, as it moves from bottom to the top of the decision-making tree. Forward chaining uses PROLOG language to deal with inference rules.

4.9.3 Forward Chaining Algorithm

As discussed, a forward-chaining algorithm starts with known facts and proceeds to trigger all the inference rules whose premises are satisfied. The new data derived from the existing one is then added to the known facts. This process is repeated till the goal is achieved or the problem is solved.

The inference engine plays an important role here. It scans the inference rules till it finds one for which the antecedent (If clause) is known to be true. Once such a rule is found, the inference engine can deduce or conclude the consequent (**Fig 4.16**) (Then clause). This results in the addition of new data.

The forward-chaining algorithm uses the following step by step process to complete a task and reach the logical or accurate conclusion.

Step 1: The system is fed with information about the problem.

Step 2: This information is stored in the working memory in the form of rules.

Step 3: The inference engine scans the rules present in the working memory in a predefined sequence to find a rule that matches contents.

Step 4: The system evaluates the rules to identify new matches.

Step 5: This process is repeated until no further matches are found.

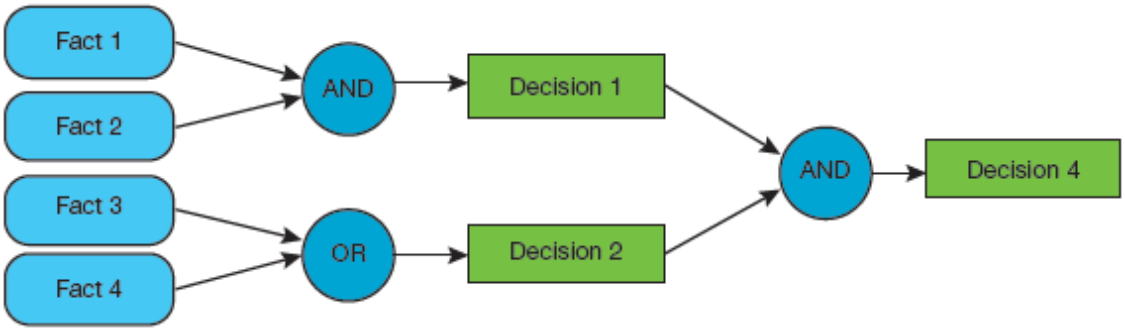


FIGURE 4.16 Forward Chaining

Example: Use the facts given below to prove that 'Raza is criminal.'

'As per the law, it is a crime for an Indian to sell weapons to enemy nations. ABC is an enemy of India and has some missiles, and all the missiles were sold to it by Raza, who is an Indian.'

The first step is to convert all the facts into first-order definite clauses.

Facts Conversion into FOL

It is a crime for an Indian to sell weapons to hostile nations.

Using variables p, q and r, we can write,

Indian (x) ∧ weapon(y) ∧ sells (x,y,z) ∧ enemy(p, India) → Criminal(x)	(i)
ABC is an enemy of India.	
This can be written as, enemy(ABC, India)	(ii)
ABC has some missiles.	
Owns(ABC, p) ∧ Missile(p)	(iii)
All of the missiles were sold to ABC by Raza.	
∀p Missiles(p) ∧ Owns (ABC, p) → Sells (Raza, p, ABC)	(iv)

Missiles are weapons.	
Missile(p) → Weapons (p)	(v)
Raza is Indian.	
Indian(Raza).	(vi)

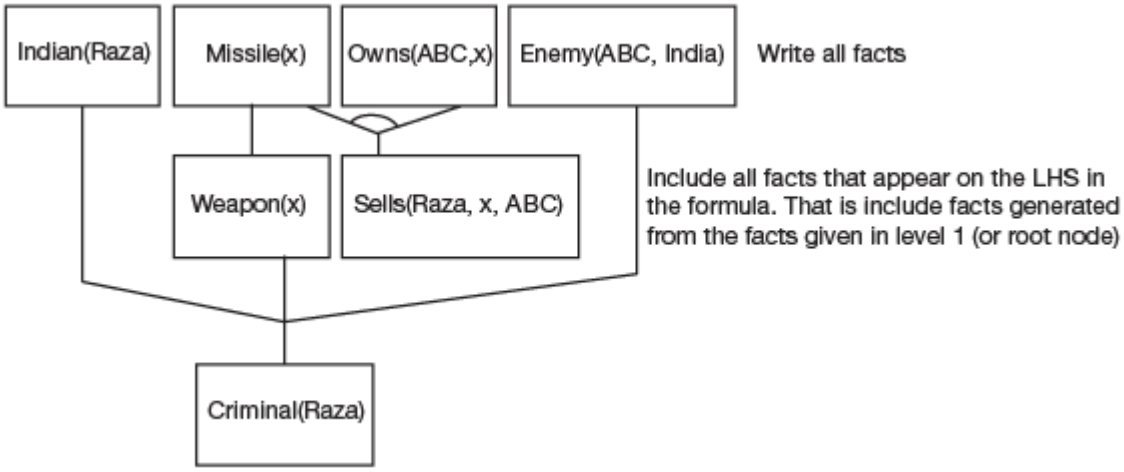


FIGURE 4.17 Forward chaining proof

Hence from **Fig. 4.17**, it is proved that Raza is a criminal using forward chaining approach.

4.9.4 Why Do We Use Forward Chaining?

Forward chaining strategy is used when the final outcome is not known, but there are facts available about the domain in which the AI system functions. Facts available in the knowledge base are used to reach the goal state or the final outcome.

For this reason, forward chaining is generally used for planning, monitoring, controlling and interpreting applications. It is an implementation strategy that is very widely used in expert systems, business and production rule systems.

Advantages of Forward Chaining

1. It is more adaptable than backward chaining as there is no restriction on the information that can be obtained from it.
2. It gives a decent premise to come to conclusions.
3. There are no limits on the data that can be derived from it. This makes it more flexible than backward chaining.
4. It can be used to draw multiple conclusions

Disadvantages of Forward Chaining

1. It takes time to synchronize and eliminate accessible information.
2. It is slower than backward chaining.
3. It generates new information without understanding what is important and relevant to the problem solution.
4. It requires lot of data as input.
5. Much of the data stored in the knowledge base may not even be relevant to the final conclusion.
6. The hypothesis performed by the forward chaining is at times not clear.
7. It might give different conclusions, which may result in ambiguity.
8. Forward chaining is good to be used on problems that have a single starting point and multiple possible endpoints. It is an inefficient algorithm when used on problems that have several starting points and just a single endpoint.

4.9.5 Backward Chaining

Backward chaining, also known as a backward deduction or backward reasoning, starts with the goal and works backward, chaining through rules to find known facts that support the goal.

It is a top-down approach that is based on modus ponens inference rule and uses a depth-first search strategy for proof. In backward chaining, the goal is broken into sub-goal(s) to prove the facts true. It is a goal-driven approach, as a list of goals decides which rules are selected and used. Backward chaining algorithm follows a process similar to that of hypoth-

esis testing in human problem-solving. The only difference is that backward chaining algorithm is focused on the required rules.

Backward chaining algorithm is used in game theory, automated theorem proving tools, inference engines, proof assistants and various other AI applications.

Example: Use the facts given below to prove that 'Raza is criminal.'

'As per the law, it is a crime for an Indian to sell weapons to enemy nations. ABC is an enemy of India and has some missiles, and all the missiles were sold to it by Raza, who is an Indian.'

The first step is to convert all the facts into first-order definite clauses.

Facts Conversion into FOL:

It is a crime for an Indian to sell weapons to hostile nations.

Using variables p, q and r, we can write,

Indian (x) ∧ I weapon(y) ∧ sells (x,y,z) ∧ enemy(p, India) → Criminal(x)	(i)

ABC is an enemy of India.

This can be written as, enemy(ABC, India)	(ii)
ABC has some missiles.	
Owns(ABC, p) ∧ Missile(p)	(iii)
All of the missiles were sold to ABC by Raza.	
∀p Missiles(p) ∧ Owns (ABC, p) → Sells (Raza, p, ABC)	(iv)
Missiles are weapons.	
Missile(p) → Weapons (p)	(v)
Raza is Indian.	
Indian(Raza).	(vi)

Backward-Chaining proof:

In backward chaining, we will start with our goal predicate (refer **Fig. 4.18**), which is Criminal(Raza), and then infer further rules. Our motive is to prove that all the facts true.



FIGURE 4.18 Backward Chaining Proof

Note that at the second level, we need to infer other facts from the goal fact which satisfy the rules. Similarly, at level three, we extract further facts that can be inferred from facts in level 2 and satisfy the rules. Now that all the rules have been proved true, we have successfully proved that Raza is a criminal using backward chaining technique.

4.9.6 Pros and Cons of Backward Chaining

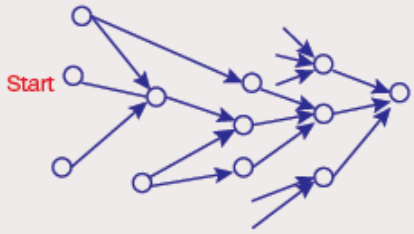
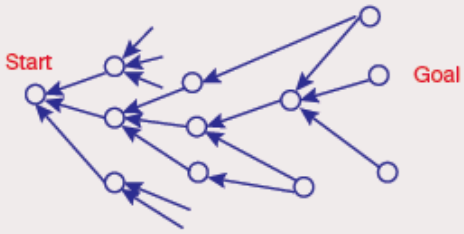
Advantages of Backward Chaining

1. If pre-decided principles are met by the inference engine, then the right arrangements can be easily inferred.
2. As compared to forward chaining, backward chaining is a faster strategy as the endpoint is accessible.
3. Since the goal is known, it becomes easier to deduce inferences.
4. It helps to solve complex problems into easy, small, fixed sections.
5. It is suitable for problems involving diagnostics, prescription and debugging.

Disadvantages of Backward Chaining

1. It just infers information that is required, which makes it less adaptable than forward chaining.
2. It cannot be used to conclude multiple solutions.
3. The path towards reasoning begins only if the goal is known.
4. It may follow a particular path of reasoning and search even though it eventually proves to be impractical.

TABLE 4.12 Forward Chaining vs Backward Chaining

Forward Chaining	Backward Chaining
It follows a bottom-up approach.	It follows a top-down approach.
It uses breadth-first search strategy.	It uses depth-first search strategy.
It works in the forward direction.	It works in the backward direction.
The technique is data-driven that uses available data to reach the goal state.	The technique is goal-driven.
Reception of new data results in new inferences, which makes the engine more suitable for dynamic situations.	It is not dynamic when compared with forward chaining.
It is used in planning, monitoring, controlling and interpreting applications.	It is employed in automated inference engines, theorem proofs, proof assistants, etc.
When there is no new data in the system, using forward chaining makes sense.	If it is a query to which a response is required, backward chaining is preferred.
Examples include XCON, the rule-based system; it is a forward chaining expert system that is used to configure large computer systems.	MYCIN is an example of backward chaining expert systems that used AI to identify severe infections caused by bacteria.
	

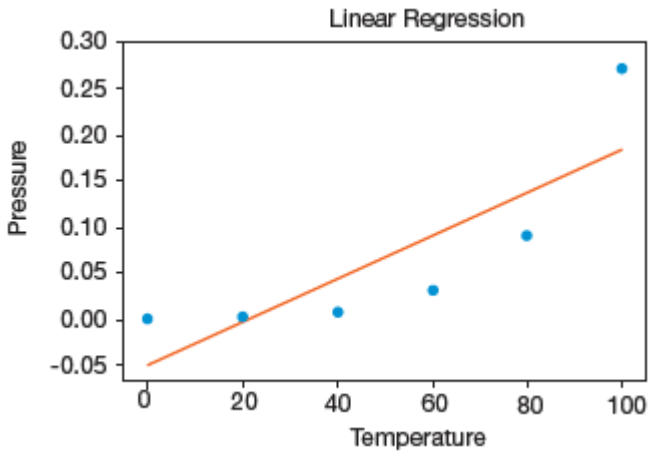
PROGRAMMER'S ZONE

Both forward chaining and backward chaining have their pros and cons. Therefore, new systems are created using hybrid chaining inference technique that deliver the advantages of both forward and backward chaining. These systems are more powerful and are capable of performing tasks that were earlier termed unachievable.

A Polynomial Regression algorithm is also called Polynomial Linear Regression because it does not depend on the variables, instead, it depends on the coefficients, which are arranged in a linear fashion.

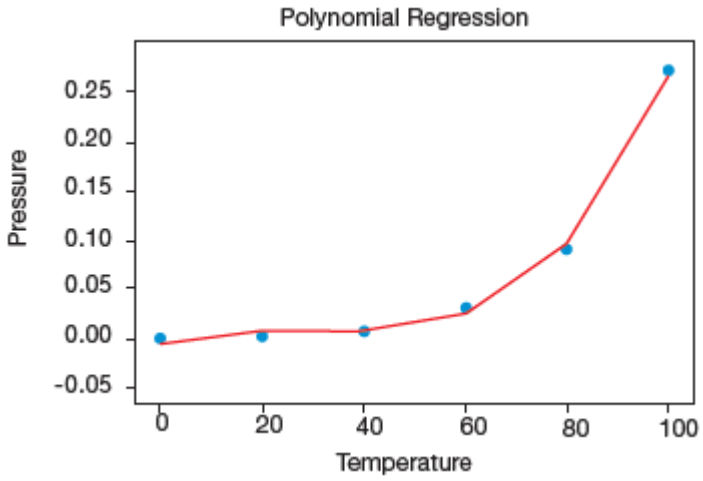
Download a sample data set from https://media.geeksforgeeks.org/wp-content/uploads/data.csv and compare the performance of linear and polynomial regression.

```
# Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
# Importing the dataset
datas = pd.read_csv('data1.csv')
print(datas )
# Dividing the dataset into 2 components
X = datas.iloc[:, 1:2].values
y = datas.iloc[:, 2].values
# Fitting Linear Regression to the dataset
# Fitting Linear Regression to the dataset
from sklearn.linear_model import LinearRegression
lin = LinearRegression()
lin.fit(X, y)
# Visualising the Linear Regression results
plt.scatter(X, y, color = 'blue')
plt.plot(X, lin.predict(X), color = 'red')
plt.title('Linear Regression')
plt.xlabel('Temperature')
plt.ylabel('Pressure')
plt.show()
```



```
# Fitting Polynomial Regression to the dataset
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree = 4)
X_poly= poly.fit_transform(X)
poly.fit(X_poly, y)
lin2 = LinearRegression()
lin2.fit(X_poly, y)
# Visualising the Polynomial Regression results
plt.scatter(X, y, color = 'blue')
plt.plot(X,  lin2.predict(X_poly),  color  =
'red')
plt.title('Polynomial Regression')
plt.xlabel('Temperature')
plt.ylabel('Pressure')
plt.show()
```

To prevent over-fitting, we can add more training samples so that the algorithm doesn't learn the noise in the system and can become more generalized.



Generate a random data set having 10 samples and 5 features. Use Ridge Regression with an alpha value =0.2 to create a model. Print the score of this model and also the value of attributes.

```
from sklearn.linear_model import Ridge
import numpy as np
n_samples, n_features = 10, 5
#np.random.RandomState() constructs a random number generator.
rng = np.random.RandomState(0)
y = rng.randn(n_samples)
X = rng.randn(n_samples, n_features)
rdg = Ridge(alpha = 0.2)
rdg.fit(X, y)
print(rdg.score(X,y))
print(rdg.coef_)
print(rdg.intercept_)
```

OUTPUT

```
0.8657654687023124
[ 1.1961396  -0.09574812 -0.96234511  0.06421247  1.66880754]
1.1808729929860098
```

Implementing lasso Regression on cancer data

```
import math
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
# difference of lasso and ridge regression is that some of the
coefficients can be zero i.e. some of the features are
# completely neglected
from sklearn.linear_model import Lasso
from sklearn.linear_model import LinearRegression
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
cancer = load_breast_cancer()
#print cancer.keys()
cancer_df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
#print cancer_df.head(3)
X = cancer.data
Y = cancer.target
X_train,X_test,y_train,y_test=train_test_split(X,Y,  test_size=0.3,
random_state=31)
# instantiate the Lasso Regression model
lasso = Lasso()
# fits the model to the training data.
lasso.fit(X_train,y_train)
train_score=lasso.score(X_train,y_train)
test_score=lasso.score(X_test,y_test)
coeff_used = np.sum(lasso.coef_!=0)
print('training score:', train_score )
print('test score: ', test_score)
print('number of features used: ', coeff_used)
```

OUTPUT

```
training score: 0.5600974529893081
test score:  0.5832244618818156
number of features used:  4
```

Implement ElasticNet Regression on the Boston Housing Data Set.

```
# import model
from sklearn.linear_model import ElasticNet
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
# Loading pre-defined Boston Dataset
boston_dataset = datasets.load_boston()
print(boston_dataset.DESCR)
# Load the dataset into Pandas Dataframe
boston_pd = pd.DataFrame(boston_dataset.data)
boston_pd.columns = boston_dataset.feature_names
boston_pd_target = np.asarray(boston_dataset.target)
boston_pd['House Price'] = pd.Series(boston_pd_target)
# input
X = boston_pd.iloc[:, :-1]
#output
Y = boston_pd.iloc[:, -1]
print(boston_pd.head())
x_train, x_test, y_train, y_test = train_test_split(
    boston_pd.iloc[:, :-1], boston_pd.iloc[:, -1],
    test_size = 0.25)
print('Train data shape of X = % s and Y = % s : '%(
    x_train.shape, y_train.shape))
print('Test data shape of X = % s and Y = % s : '%(
    x_test.shape, y_test.shape))
# Train the model
e_net = ElasticNet(alpha = 1)
e_net.fit(x_train, y_train)
# calculate the prediction and mean square error
y_pred_elastic = e_net.predict(x_test)
mean_squared_error = np.mean((y_pred_elastic - y_test)**2)
print('Mean Squared Error on test set', mean_squared_error)
e_net_coeff = pd.DataFrame()
e_net_coeff['Columns'] = x_train.columns
e_net_coeff['Coefficient Estimate'] = pd.Series(e_net.coef_)
print('COEFFICIENT = ', e_net_coeff)
```

OUTPUT

```
      CRIM    ZN  INDUS  CHAS ...  PTRATIO       B  LSTAT  House Price
0  0.00632  18.0   2.31   0.0 ...     15.3  396.90   4.98         24.0
1  0.02731   0.0   7.07   0.0 ...     17.8  396.90   9.14         21.6
2  0.02729   0.0   7.07   0.0 ...     17.8  392.83   4.03         34.7
3  0.03237   0.0   2.18   0.0 ...     18.7  394.63   2.94         33.4
4  0.06905   0.0   2.18   0.0 ...     18.7  396.90   5.33         36.2
[5 rows x 14 columns]
Train data shape of X = (379, 13) and Y = (379,) :
Test data shape of X = (127, 13) and Y = (127,) :
Mean Squared Error on test set 23.968225718955683
COEFFICIENT =      Columns  Coefficient Estimate
0       CRIM            -0.073887
1         ZN             0.037001
2      INDUS            -0.025703
3       CHAS             0.000000
4        NOX            -0.000000
5         RM             0.855715
6        AGE             0.029336
7        DIS            -0.711843
8        RAD             0.372780
9        TAX            -0.019305
10   PTRATIO            -0.777579
11         B             0.010634
12     LSTAT            -0.794296
```

Implement binary logistic regression on random data values. Calculate the accuracy of the model and plot the confusion matrix. Set the value of C = 10 and check whether the model performs better or not.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np
# array of consecutive, equally-spaced values within a given range
x = np.arange(10).reshape(-1, 1) # make x a 2D array
# have a value for each observation in x
y = np.array([0, 0, 0, 0, 1, 1, 1, 1, 1, 1])
model = LogisticRegression()
model.fit(x, y)
print('model.classes_ = ', model.classes_)
print('model.intercept = ', model.intercept_)
print('model.coef = ', model.coef_)
# to get matrix of probabilities that the predicted output is equal
to zero: 1 - p(x) or one: p(x)
p_pred = model.predict_proba(x)
y_pred = model.predict(x)       # predict y values
score_ = model.score(x, y)   # accuracy of the model
conf_m = confusion_matrix(y, y_pred)
# to get a more comprehensive report on the classification
report = classification_report(y, y_pred)
print('p_pred = ', p_pred)
print('y_pred = ', y_pred)
print('Accuracy- ratio of the number of correct predictions to the
number of observations
= ', score_)
print('conf_m = ', conf_m)
print('report = ', report)
```

OUTPUT

```
model.classes_ =  [0 1]
model.intercept =  [-1.04608067]
model.coef =  [[0.51491375]]
p_pred =  [[0.74002157 0.25997843]
 [0.62975524 0.37024476]
 [0.5040632  0.4959368 ]
 [0.37785549 0.62214451]
 [0.26628093 0.73371907]
 [0.17821501 0.82178499]
 [0.11472079 0.88527921]
 [0.07186982 0.92813018]
 [0.04422513 0.95577487]
 [0.02690569 0.97309431]]
y_pred =  [0 0 0 1 1 1 1 1 1 1]
score_ =  0.9
conf_m =  [[3 1]
 [0 6]]
report =       precision    recall  f1-score   support

           0       1.00      0.75      0.86         4
           1       0.86      1.00      0.92         6

    accuracy                           0.90        10
   macro avg       0.93      0.88      0.89        10
```



KEY TERMS

Meta-knowledge: It is knowledge about knowledge.

Heuristic knowledge: **It refers to k**nowledge about a specific topic.

Procedural knowledge or imperative knowledge: It refers to knowledge that gives information about how to achieve or do something. This knowledge includes rules, strategies, procedures, agendas, etc. that can be directly applied to perform any task.

Declarative knowledge: This is the information that we have about an object. This knowledge helps us to describe a particular concept, fact, object and its attributes.

Structural Knowledge: This is the basic knowledge to solve complex problems which describes relationships between various concepts or objects such as kind of, part of, and grouping of something.

Logic: It is the main component of any knowledge that facilitates a person to draw conclusion by filtering required information from large volumes of sentences.

Frame: It is a record-like structure that describes a real world using a collection of attributes and values.

Proposition: It is a declarative statement which is either true or false but it cannot be both (true as well as false).

Atomic proposition: It is a simple proposition made of a single proposition symbol. Such propositions can be either true or false.

Compound proposition: These propositions are formed by combining simpler atomic propositions, using parenthesis and logical operators.

Inference: It is the process of generating conclusions from evidence and facts.

Inference rules: These are the templates for generating valid arguments that are used to derive proofs to obtain conclusions that finally lead to the desired goal.

Free Variable: It is a variable in a formula that occurs outside the scope of the quantifier.

Bound Variable: It is a variable in a formula that occurs within the scope of the quantifier.

Skolemization: Eliminating existential instantiation quantifier (∃) by replacing it with skolem constant.

Definite clause: This is a clause which is a disjunction of literals with exactly one positive literal.

Horn clause: This is a clause which is a disjunction of literals with at most one positive literal.



RECAPITULATION POINTS

- Knowledge representation and reasoning (KR, KRR) is a part of artificial intelligence which focuses on how AI agents think and how their thinking contributes to intelligent behaviour.
- A knowledge-based system represents information about the real world in such a way that a computer can easily understand and utilize this knowledge to solve complex real-world problems.
- Knowledge-based agents represent knowledge that can be reasoned to act intelligently according to the requirements. Usually, these agents store knowledge about their surroundings in the form of sentences.
- The inference system derives new sentences from old ones and also facilitates addition of new sentence(s) to the knowledge base.
- Logical representation is a language with some definite rules.
- The production rules system consists of (condition, action) pairs which mean, ‘If condition then action’.
- At times, it happens that multiple production rules are fired together. This situation is called conflict set. In such a situation, the agent needs

to select one particular rule. This is often referred to as conflict resolution.

- While universal quantifier, represented by ∀, specifies that the statement (within its range) is true for everything or every instance of a particular thing, existential quantifiers (denoted as ∃) express that the statement within its scope is true for at least one instance of something.
- The generalized modus ponens states that if P implies Q and P is asserted to be true, then Q must be true.
- The unification process is used to make two different logical atomic expressions (or literals) identical by finding a substitution. Resolution is usually used when there are multiple statements and we have to prove a conclusion.
- Forward chaining starts with atomic sentences in the knowledge base and applies inference rules (modus ponens) in the forward direction to extract more data until a goal is reached.
- Backward-chaining starts with the goal and works backward, chaining through rules to find known facts that support the goal.



ANSWER THE QUESTIONS

1. What knowledge needs to be represented in an AI system?
2. Broadly classify the types of knowledge.
3. Define logic and give its main components.
4. Explain the components of an AI system.
5. Give the properties of a knowledge representation system.
6. What are knowledge-based agents? Explain their role and architecture.
7. With the help of relevant examples, state the operations performed by a knowledge-based agent.
8. What are the different levels at which a knowledge-based agent can be viewed?
9. Explain the different techniques of representing knowledge.
10. What do you understand by recognize-act cycle and conflict resolution?
11. What different logical connectives are used in propositional logic?

12. Explain the basic rules in First Order Logic.
13. Give the conditions that must be satisfied for unification process.
14. List the steps to be performed in a resolution process.
15. Represent the following statements in the form of nodes and arcs.
    Statements:
    1. Jerry is a cat.
    2. Jerry is a mammal.
    3. Jerry is owned by Priya.
    4. Jerry is brown coloured.
    5. All mammals are animals.
16. Consider an entity, Achyut, who is a 25-year-old engineer by profession and lives in Rameshwaram, India. Design a frame to record this information.
17. Write the rules for the following statements.
    1. If it rains today and one does not carry umbrella, he will be drenched.
    2. If it rains today and Arjun does not carry umbrella, then he will be drenched.
18. Can the following statements be represented using propositional logic? Justify your answer.
    1. ‘Earth is round’
    2. ‘Some children are lazy’
    3. ‘Garlic tastes wonderful’
19. If, P = Aakash goes to temple and Q = Aakash is religious
    1. Prove using Modus Tollen that Akash does not go to temple.
    2. What does Modus Ponen say in this case?
20. Given the facts, if Sheero is intelligent, then Sheero is smart. Sheero is intelligent. Prove that Sheero is smart.
21. If Ram is the friend of Shyam and Shyam is the friend of Rahul, then prove that Ram is the friend of Rahul.
22. Represent the fact, ‘Myra is not beautiful or she is obedient’.
23. Given the facts, ‘All human beings are mortal.’ ‘Sachin is a human being.’ Prove that ‘Sachin is mortal’.
24. Let P be ‘it snows.’ Let Q be ‘I will study.’ Let R be ‘I will get an A grade.’
    This can conclude that ‘If it snows, then I will study’ and ‘If I study, I will get an A grade.’ Therefore, ‘If it snows, I will get an A grade.’

Using which rule, can we get this conclusion? Explain.

25. Name the tautology and give an example.
    1. ((p) ∧ (q)) →(p ∧ q)
    2. ((p ∨ q) ∧ (¬p ∨ r)) →(q ∨ r)
    3. ((p → q) ∧ (r → q) ∧ (p ∨ r)) → q
    4. ((p → q) ∧ (r → s) ∧ (p ∨ r)) → (q ∨ s)
    5. (p → q) ∧ (r → s) ∧ (¬q ∨ ¬s) → (¬p ∨ ¬r)
26. Given the assumptions, prove by resolution.
    1. Assumptions
        1. Fido is a dog
        2. All dogs are animals
        3. All animals will die

        Goal- Fido will die.
    2. Axioms
        1. allergies(X) → sneeze(X)
        2. cat(Y) ∧ allergic-to(X,Y) → allergies(X)
        3. cat(Felix)
        4. allergic-to(Liz, Felix)

        Goal: sneeze(Liz)
    3. Assumptions
        1. All hounds howl at night.
        2. Anyone who has any cat will not have any mice.
        3. Light sleepers do not have anything which howls at night.
        4. John has either a cat or a hound.

        Goal: If John is a light sleeper, then John does not have any mice.
    4. Assumptions
        1. Every Austinite who is not conservative loves some armadillo.
        2. Anyone who wears maroon-and-white shirts is an Aggie.
        3. Every Aggie loves every dog.
        4. Nobody who loves every dog loves any armadillo.
        5. Clem is an Austinite, and Clem wears maroon-and-white shirts.

        Goal: Is there a conservative Austinite?
    5. Assumptions
        1. Anyone whom Mary loves is a football star.
        2. Any student who does not pass does not play.
        3. John is a student.
        4. Any student who does not study does not pass.

5. Anyone who does not play is not a football star.

Goal: If John does not study, then Mary does not love John.

6. Assumptions
    1. Every coyote chases some roadrunner.
    2. Every roadrunner who says “beep-beep” is smart.
    3. No coyote catches any smart roadrunner.
    4. Any coyote who chases some roadrunner but does not catch it is frustrated.

    Goal: If all roadrunners say “beep-beep”, then all coyotes are frustrated.
7. Assumptions
    1. Anyone who rides any Harley is a rough character.
    2. Every bike rider [something that is] either a Harley or a BMW.
    3. Anyone who rides any BMW is a yuppie.
    4. Every yuppie is a lawyer.
    5. Any nice girl does not date anyone who is a rough character.
    6. Mary is a nice girl, and John is a biker.

    Goal: If John is not a lawyer, then Mary does not date John.
8. Assumptions
    1. Every child loves anyone who gives the child any present.
    2. Every child will be given some present by Santa if Santa can travel on Christmas eve.
    3. It is foggy on Christmas eve.
    4. Anytime it is foggy, anyone can travel if he has some source of light.
    5. Any reindeer with a red nose is a source of light.

    Goal: If Santa has some reindeer with a red nose, then every child loves Santa.
9. Assumptions
    1. Every investor bought [something that is] stocks or bonds.
    2. If the Dow-Jones Average crashes, then all stocks that are not gold stocks fall.
    3. If the T-Bill interest rate rises, then all bonds fall.
    4. Every investor who bought something that falls is not happy.

    Goal: If the Dow-Jones Average crashes and the T-Bill interest rate rises, then any investor who is happy bought some gold stock.
10. Assumptions

1. Every child loves every candy.
2. Anyone who loves some candy is not a nutrition fanatic.
3. Anyone who eats any pumpkin is a nutrition fanatic.
4. Anyone who buys any pumpkin either carves it or eats it.
5. John buys a pumpkin.
6. Lifesavers is a candy.

Goal: If John is a child, then John carves some pumpkin.

27. Unify the following pairs of predicates and give the resulting substitutions.

    Assumptions

    1. Every child loves Santa.
    2. Everyone who loves Santa loves any reindeer.
    3. Rudolph is a reindeer, and Rudolph has a red nose.
    4. Anything which has a red nose is weird or is a clown.
    5. No reindeer is a clown.
    6. Scrooge does not love anything which is weird.

    Goal: Scrooge is not a child.

28. Write short notes on the following:
    1. Inference rules
    2. First order logic
        1. Differentiate between the following:
            1. Declarative and procedural approach of designing KBA
            2. Universal and existential quantifier
            3. Free and bound variables
            4. Forward and backward chaining



FILL UP

1. An intelligent agent acts by _____ its environment.
2. If we remove the _____ component, then the decision-maker will not be able to sense the environment accurately and take appropriate decisions.
3. Knowledge representation models intelligent behaviour for an _____.
4. The knowledge about knowledge is known as _____.
5. In AI, knowledge is represented using _____.

6. _____ uses inference algorithms to think all the possible reasons that can give a proper result.
7. _____ efficiency helps the knowledge representation system to easily acquire new knowledge.
8. Knowledge-based agents store knowledge about their surroundings in the form of _____.
9. _____ is the knowledge-based system engine that infers new knowledge in the system.
10. _____ represents knowledge through graphical networks.
11. _____ system consists of (condition, action) pairs which mean, 'If condition then action'.
12. _____ is a declarative statement which is either true or false but it cannot be both.
13. A proposition formula which is always true is called _____.
14. A proposition formula which is always false is called _____.
15. Two propositions are said to be _____ if and only if the columns in the truth table are identical to each other.
16. Law of associativity states that (P ∨ Q) ∨ R= _____.
17. Statements which cannot be answered absolutely are called _____ statements.
18. The process of generating conclusions from evidence and facts is known as _____.
19. The negation of implication is called _____ which is represented as _____.
20. The _____ syllogism rule states that if P∨Q is True, and ¬P is True, then Q will be true.
21. A complex sentence in FOL has two main parts— _____ and _____.
22. _____ specifies the quantity of specimen in the universe of discourse.
23. _____ operator specifies that the two terms refer to the same object.
24. The _____ states that if P implies Q and P is asserted to be true, then Q must be True.
25. The substitution variables are called _____.
26. _____ process is used to make two different logical atomic expressions identical by finding a substitution.
27. _____ is a sentence represented as a conjunction of clauses.
28. A clause which is a disjunction of literals with exactly one positive literal is known as a _____ clause.

29. _____ chaining starts with atomic sentences in the knowledge base and applies inference rules (Modus Ponens) in the forward direction to extract more data until a goal is reached.
30. If today is Monday is represented as P, then today is not a Monday will be written as _____.
31. If Ram is a cricket player is written as P and Ram is a hockey player is represented as Q, then Ram plays both cricket and hockey is represented by _____.
32. If Ram leaves for Mumbai is represented by P and Ram leaves for Chennai is written as Q, then Ram leaves for Chennai or Mumbai is represented by _____.
33. The clause if it is Sunday (P), then I will go to Movie (Q), is represented as _____.
34. If I have Rs 500 then only I will buy the book. The converse condition that I will buy this book if and only if I have Rs 500 is represented as _____.
35. It rains in July can be written as _____.



STATE TRUE OR FALSE

1. Logic is the main component of any knowledge that facilitates a person to draw conclusion by filtering required information from large volumes of sentences.
2. Syntax is used to check if a syntactically correct sentence is logically correct and meaningful or not.
3. Inferential adequacy directs the inferential knowledge mechanism to generate appropriate results.
4. Learning element of the KBA updates the knowledge base every few days.
5. In the procedural approach, the desired behaviour is directly encoded in the agent as a program code.
6. Propositional Logic (FoL) represents the objects in quantifiers and predicates.
7. It is impossible to represent all facts from human's memory into a semantic network.

8. Statements which are questions, commands or opinions are not propositions.
9. The semantics of propositional logic defines the allowable sentences for the knowledge representation.
10. (P ∧ Q) ∧ R= P ∧ (Q ∧ R) represents the law of distributive.
11. ¬ (P ∧ Q) = (¬P) ∨ (¬Q)
12. Every relationship can be represented using propositional logic.
13. Subject is a relation which binds two atoms together in a statement.
14. In existential quantifier, AND or conjunction symbol (∧) is used as the main connective.
15. A free variable occurs within the scope of the quantifier.
16. Universal instantiation is an inference rule in first-order logic that can be applied only once to replace the existential sentence.
17. The unification process will fail if two similar variables are present in the same expression.
18. A clause which is a disjunction of literals with at most one positive literal is known as definite clause.
19. Forward chaining can be used to draw multiple conclusions.



MULTIPLE CHOICE QUESTIONS

1. _____ represents information and facts about actions which occur in the real world.
    1. Objects
    2. Events
    3. Performance
    4. Meta-knowledge
2. _____ knowledge helps us to describe a particular concept, fact, object and its attributes.
    1. Meta
    2. Procedural
    3. Declarative
    4. Structural
3. _____ component retrieves data from the environment, discovers the source(s) of noise, checks if the AI was damaged by anything and defines response to be given when any sense has been detected.

    1. Perception
    2. Learning
    3. Planning and Execution
    4. Knowledge representation and reasoning
4. With _____ component, the system focuses on self-improvement through knowledge acquisition, inference, acquisition of heuristics, faster searches, etc.
    1. Perception
    2. Learning
    3. Planning and execution
    4. Knowledge representation and reasoning
5. Which of the following is not a function planning and execution component?
    1. Select an initial state
    2. Enumerate preconditions and effects
    3. Enumerate a sequence of actions to achieve the goal state
    4. Reasoning and suggestions for self-improvement
6. _____ manipulates the representational structures to produce new knowledge corresponding to the existing structure.
    1. Representational accuracy
    2. Inferential adequacy
    3. Inferential efficiency
    4. Acquisitional efficiency
7. _____ derives new sentences from old ones and also facilitates addition of new sentence(s) to the knowledge base.
    1. Knowledge representation system
    2. Knowledge base system
    3. Inference engine
    4. Knowledge-based agent
8. _____ operation informs the knowledge base which action was selected and performed by the agent.
    1. ASK
    2. TELL
    3. PERFORM
    4. Any of these
9. Sentences are encoded into different logics at _____ level.
    1. Knowledge

2. Logical
3. Implementational
4. All of these

10. _____ representation is a language with some definite rules.
    1. Logical
    2. Frame
    3. Semantic
    4. All of these
11. We can express a propositional logic using _____ operators.
    1. Arithmetic
    2. Relational
    3. Logical
    4. Membership
12. Identify the incorrect statement.
    1. An atomic proposition is said to be satisfiable if there is an interpretation for which it is true.
    2. A contingent statement is one which is neither a tautology nor a contradiction.
    3. Modus Tollens rule states that if P → Q is True and ¬ Q is True, then ¬ P will also true.
    4. The Disjunctive Syllogism rule states that P→R is True iff P→Q is true, and Q→R is true.
13. A sentence of the form P ⇔ Q represents _____
    1. Negation
    2. Disjunction
    3. Implication
    4. Bi-directional
14. P ∧ Q= Q ∧ P denotes the law of _____.
    1. Commutativity
    2. Associativity
    3. Identity Element
    4. Distributive
15. P ∧ True = _____.
    1. 0
    2. 1
    3. P
    4. True

16. _____ are the templates for generating valid arguments that are used to derive proofs to obtain conclusions that finally leads to the desired goal.
    1. Inferences
    2. Inference rules
    3. Compound proposition
    4. Implications
17. _____ is an inference rule that states if P is True and P → Q is True, then we can infer that Q will also be true.
    1. Modus Ponens
    2. Modus Tollens
    3. Hypothetical syllogism
    4. Disjunctive syllogism
18. In universal quantifier, _____ is used as the main connective.
    1. Negation
    2. Disjunction
    3. Implication
    4. Bi-directional
19. Which of the following is not similar?
    1. ∀x∀y and ∀y∀x
    2. ∃x∃y and ∃y∃x
    3. ∃x∀y and ∀y∃x
    4. None of these
20. _____ allows us to infer any sentence P(c) by substituting a ground term, c for the variable.
    1. Universal Generalization
    2. Universal Instantiation
    3. Existential Instantiation
    4. Existential introduction
21. _____ is usually used when there are multiple statements and we have to prove a conclusion.
    1. Unification
    2. Substitution
    3. Resolution
    4. Instantiation
22. Resolution technique is used in _____ logic.
    1. Propositional

2. First-order predicate
3. Both of these
4. None of these

23. Identify the correct statement about forward chaining.
    1. It is said to be data-driven approach.
    2. It works using bottom-up approach.
    3. It uses PROLOG language to deal with inference rules.
    4. All of these
24. Identify the incorrect statements.
    1. Forward chaining is slower than backward chaining.
    2. Forward chaining might give different conclusions, which may result in ambiguity.
    3. Backward chaining uses bottom-up approach.
    4. Backward chaining uses a depth-first search strategy for proof.



ANSWERS

Fill Up

1. Sensing
2. Knowledge
3. agent
4. Meta-knowledge
5. Logics
6. logical inference
7. Acquisitional
8. sentences
9. Inference engine (IE)
10. Semantic network
11. Production rules
12. Proposition
13. tautology
14. Contradiction.
15. logically equivalent
16. P ∨ (Q ∨ R)
17. Open

18. Inference
19. Inverse, ¬ P → ¬ Q.
20. Disjunctive
21. subject and predicate
22. Quantification
23. Equality
24. Generalized Modus Ponens
25. Most general unifier
26. Unification
27. Conjunctive normal form
28. Definite
29. Forward
30. ¬ P
31. P ∧ Q
32. P v Q
33. P → Q
34. P ⇔ Q
35. Rains (July)

State True or False

1. True
2. False
3. False
4. False
5. True
6. False
7. True
8. True
9. False
10. False
11. True
12. False
13. False
14. True
15. False
16. False

17. True
18. False
19. True

Multiple Choice Questions

1. b
2. c
3. a
4. b
5. d
6. b
7. c
8. b
9. b
10. a
11. c
12. d
13. d
14. a
15. c
16. b
17. a
18. c
19. c
20. b
21. c
22. c
23. d
24. c